Neural Networks and Deep Learning

Neural networks and computer vision

ECE 116 | Fall 2026

Adapted from Farias and Ramakrishnan, MIT OpenCourseWare 15.773, Spring 2024. Changes were made. CC BY-NC-SA 4.0.

OK, let's start at the beginning. What's a Neural Network?

A binary classifier



$$Pr(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

The classifier as a network

$$Pr(Y = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

Let us look at it through the lens of a “network” of mathematical operations.

We will work with this simple classification example

- Given independent variables …
  - GPA
  - Experience
- … predict who will be called for a job interview

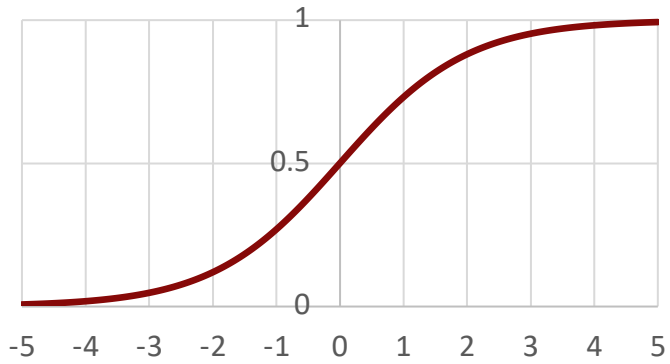
	Interview	GPA	Experience
1	0	3.27	1.93
2	0	3.37	0.07
3	0	3.57	1.91
4	0	3.91	4.35
5	0	3.20	1.70
6	1	3.90	2.41
7	1	3.94	3.00
8	0	3.66	2.47
9	0	3.63	0.93
10	0	3.06	4.14
11	0	3.21	3.34
12	0	3.18	3.97
13	0	3.69	0.54
14	0	3.38	3.62
15	1	3.77	2.06
16	0	3.50	4.10
...	...	...	...
25	1	3.31	3.46
26	0	3.78	0.29
27	0	3.87	1.21
28	0	4.00	0.49
29	1	3.87	2.11

We will work with this simple classification example

	Interview	GPA	Experience
1	0	3.27	1.93
2	0	3.37	0.07
3	0	3.57	1.91
4	0	3.91	4.35
5	0	3.20	1.70
6	1	3.90	2.41
7	1	3.94	3.00
8	0	3.66	2.47
9	0	3.63	0.93
10	0	3.06	4.14
11	0	3.21	3.34
12	0	3.18	3.97
13	0	3.69	0.54
14	0	3.38	3.62
15	1	3.77	2.06
16	0	3.50	4.10
...	...	...	...
25	1	3.31	3.46
26	0	3.78	0.29
27	0	3.87	1.21
28	0	4.00	0.49
29	1	3.87	2.11

We can estimate this logistic regression model and find values for the coefficients:

$$P(Y=1) = \frac{1}{1+e^{-(0.4+0.2\cdot \text{GPA}+0.5\cdot \text{Experience})}}$$

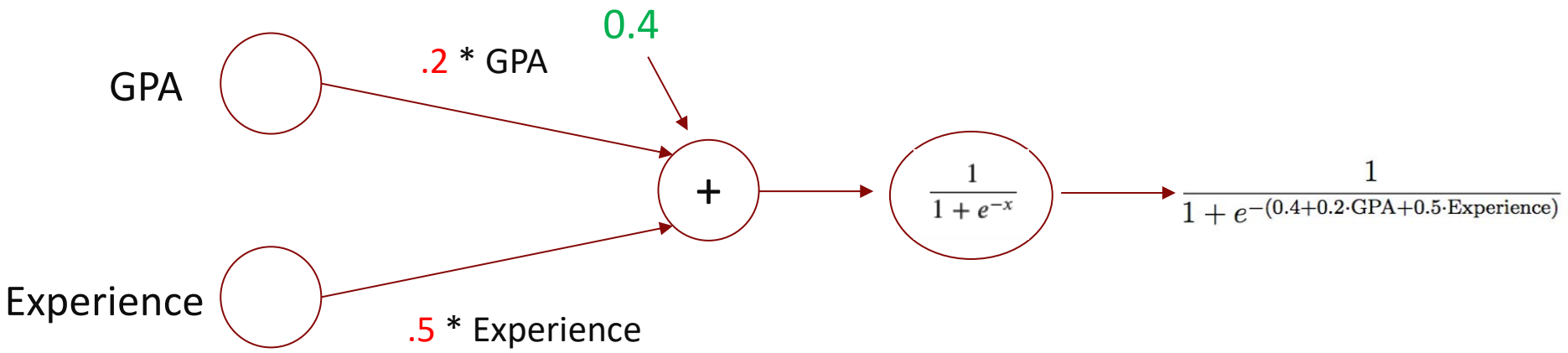


We can re-write this formula as a network with input data flowing through two functions that have been connected

Model equation: $$P(Y=1) = \frac{1}{1+e^{-(0.4+0.2\cdot \text{GPA}+0.5\cdot \text{Experience})}}$$

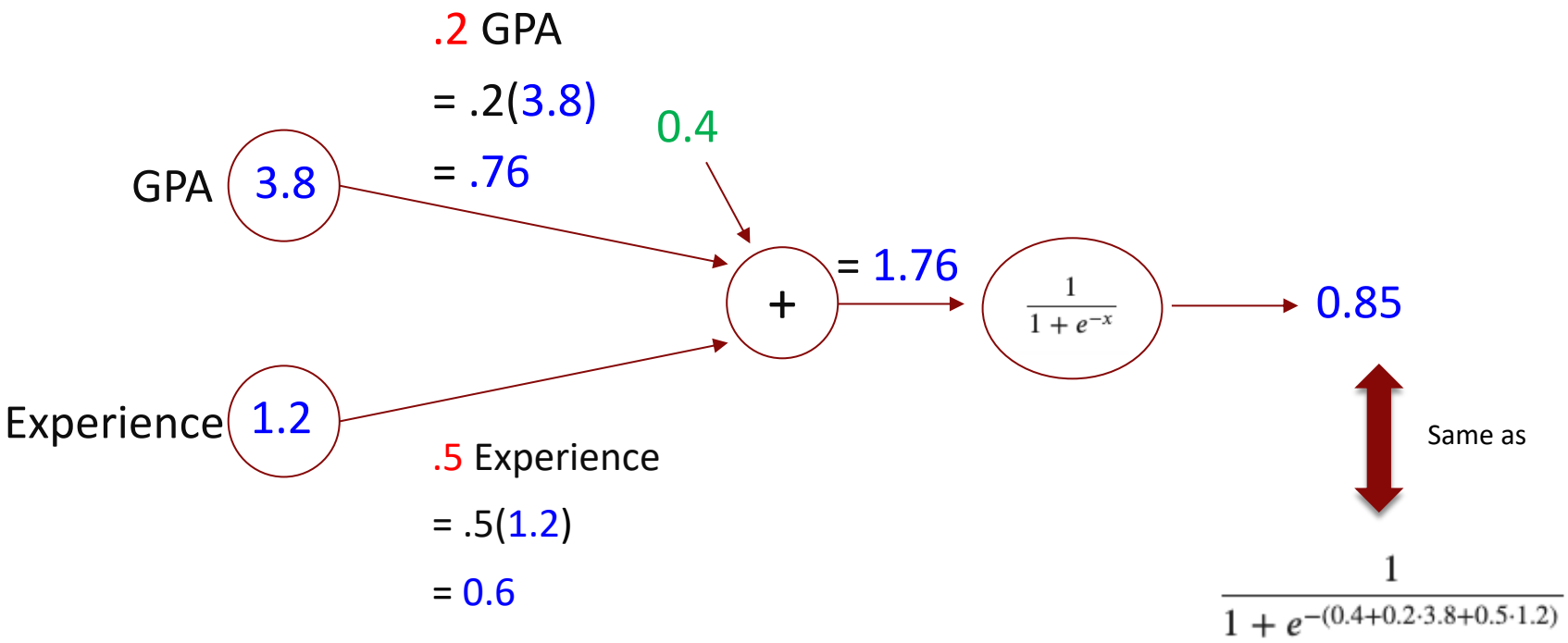
We can re-write this formula as a network with input data flowing through two functions that have been connected

Model equation: $$P(Y=1) = \frac{1}{1+e^{-(0.4+0.2\cdot \text{GPA}+0.5\cdot \text{Experience})}}$$



Let's make a prediction with this "network"

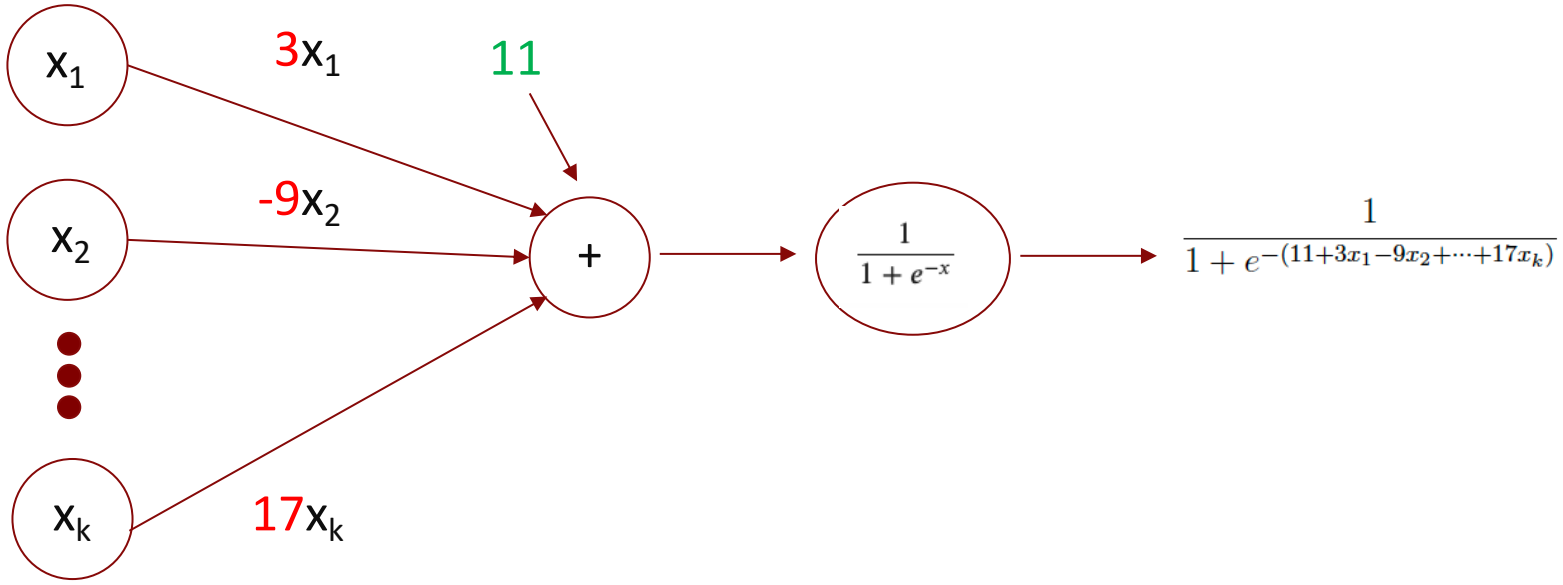
Consider a job applicant with a GPA of 3.8 and 1.2 years of experience.



The general logistic regression model viewed through a network lens

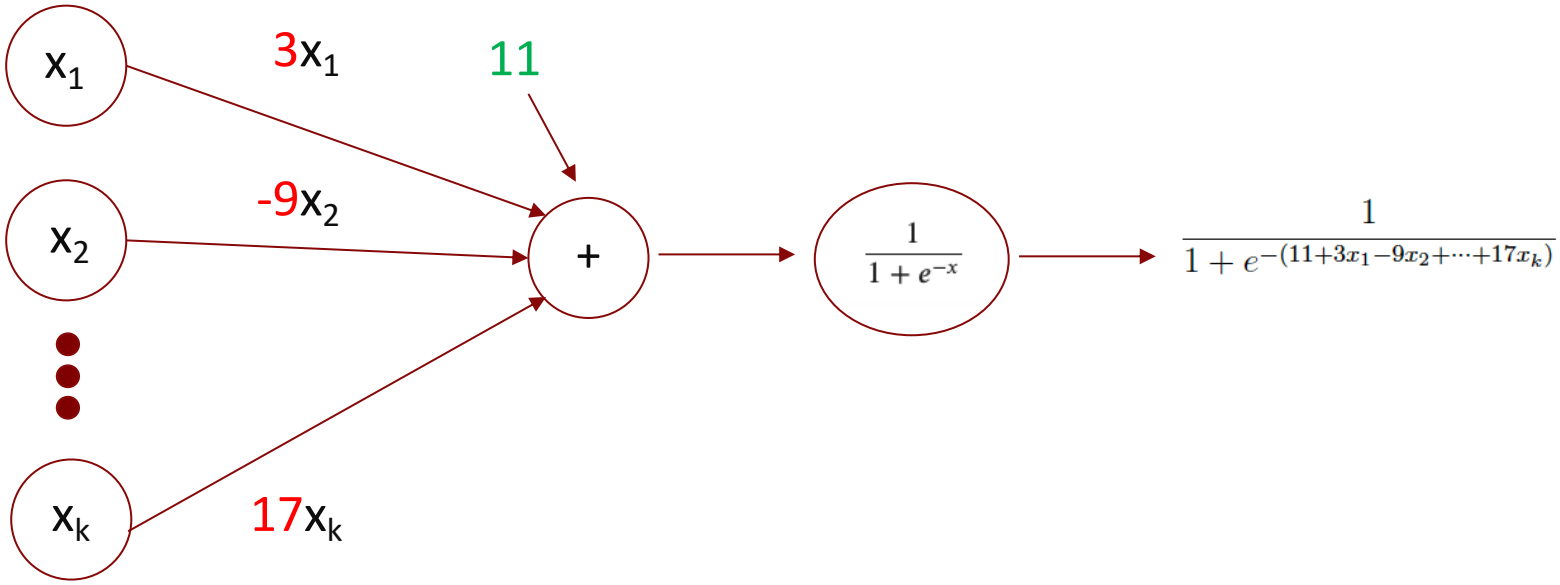
Model equation:

$$P(Y=1) = \frac{1}{1+e^{-(11+3x_1-9x_2+\cdots+17x_k)}}$$



Notice how the data flows through the network from left to right

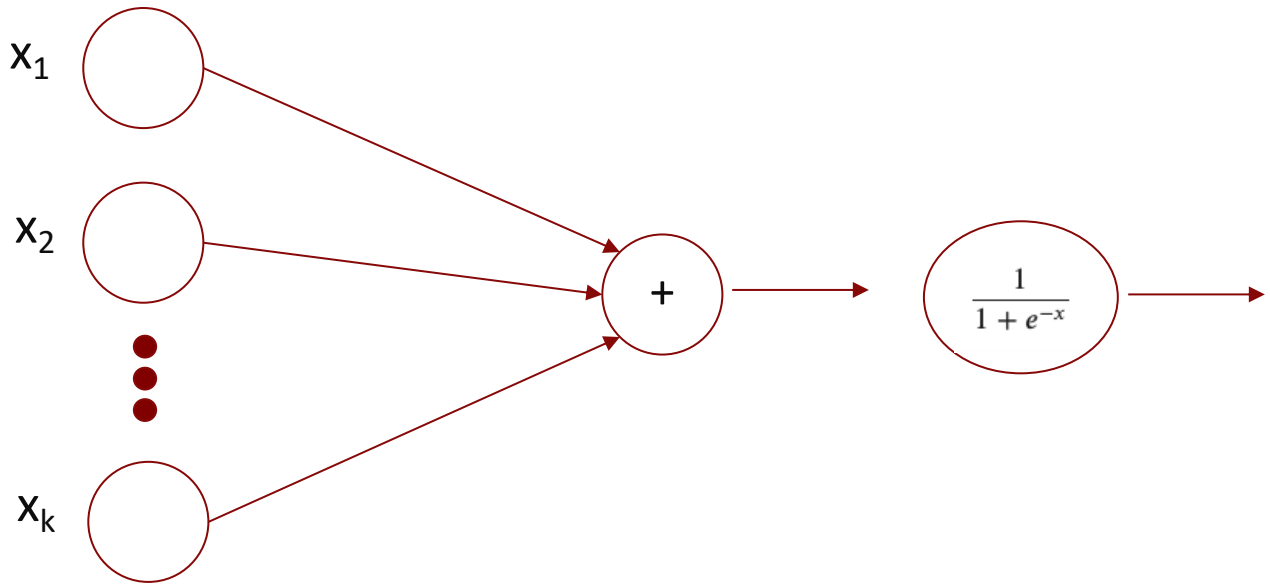
Terminology



- Multipliers on values from each node = coefficients = weights
- Intercept = bias

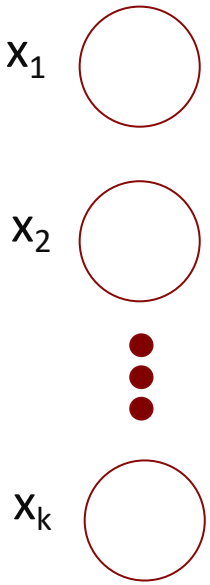
What's the advantage of viewing through a network "lens"?

Recall the notion of learning smart representations of the input data

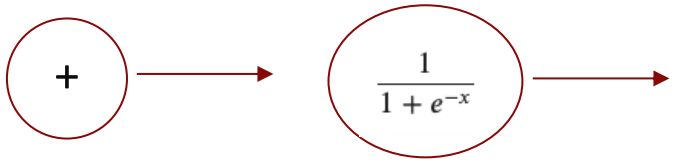


*I am not showing the weights and biases to avoid clutter

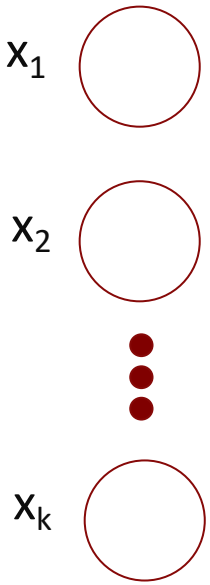
To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



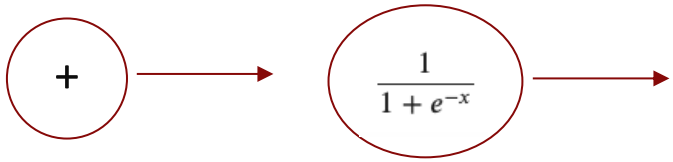




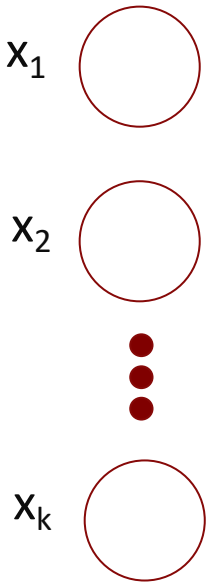
To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



What is the simplest thing we can do here?

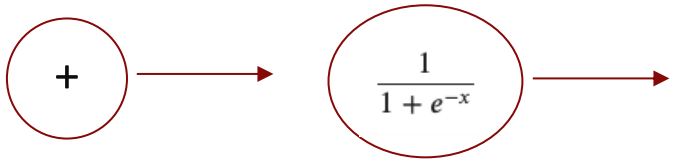


To learn smart representations, we would like to transform the inputs one or more times before we do the prediction

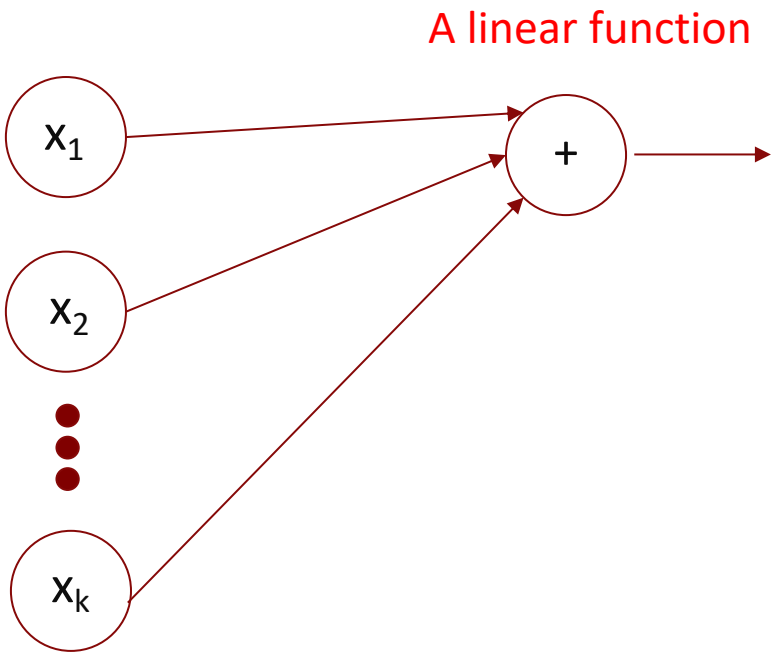


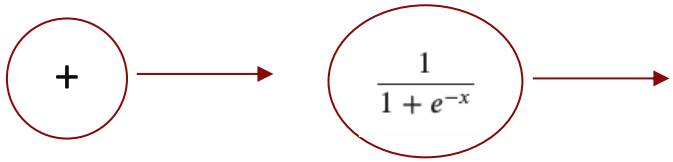
What is the simplest thing we can do here?

We can insert linear functions

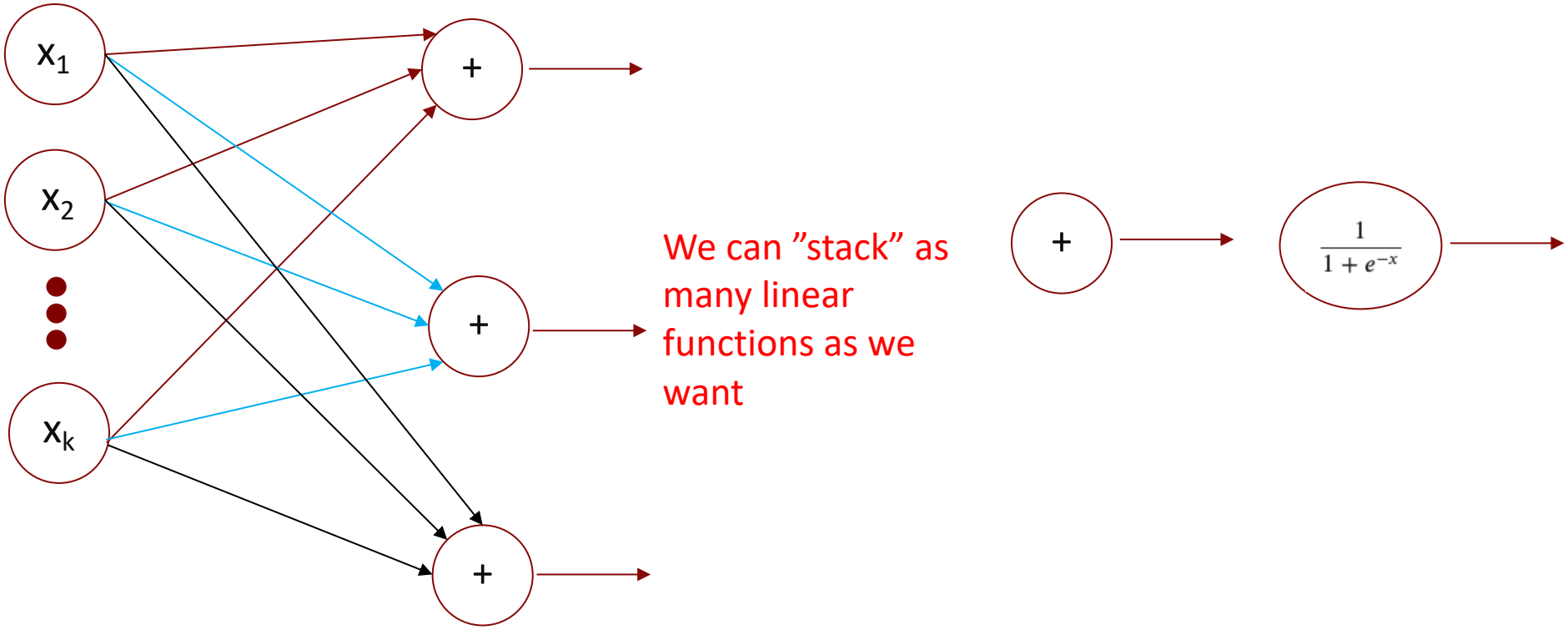


To learn smart representations, we would like to transform the inputs one or more times before we do the prediction

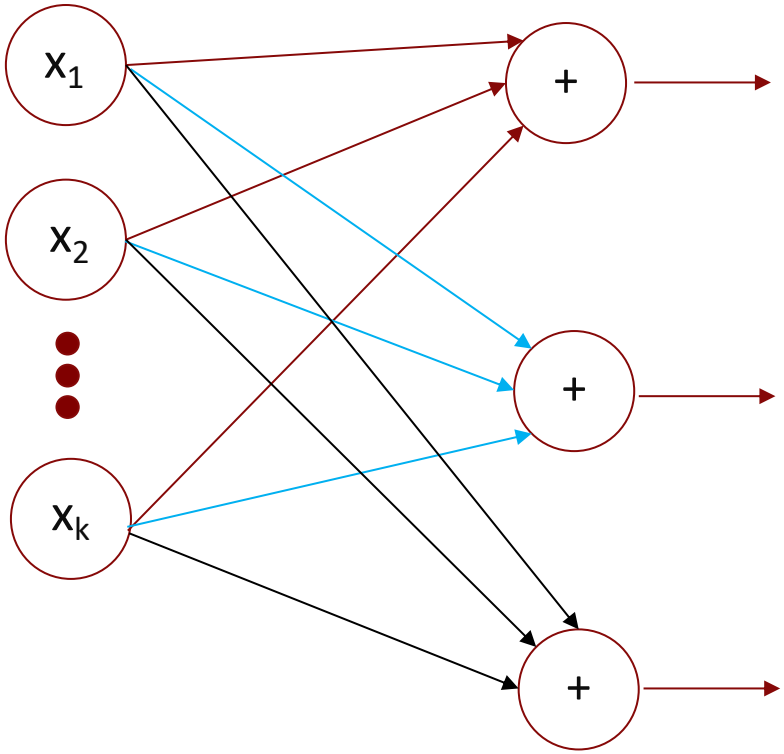




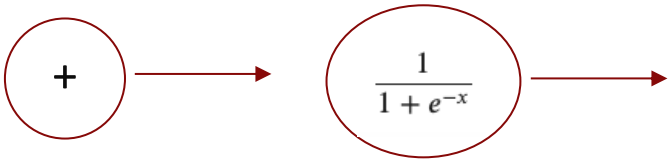
To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



To learn smart representations, we would like to transform the inputs one or more times before we do the prediction

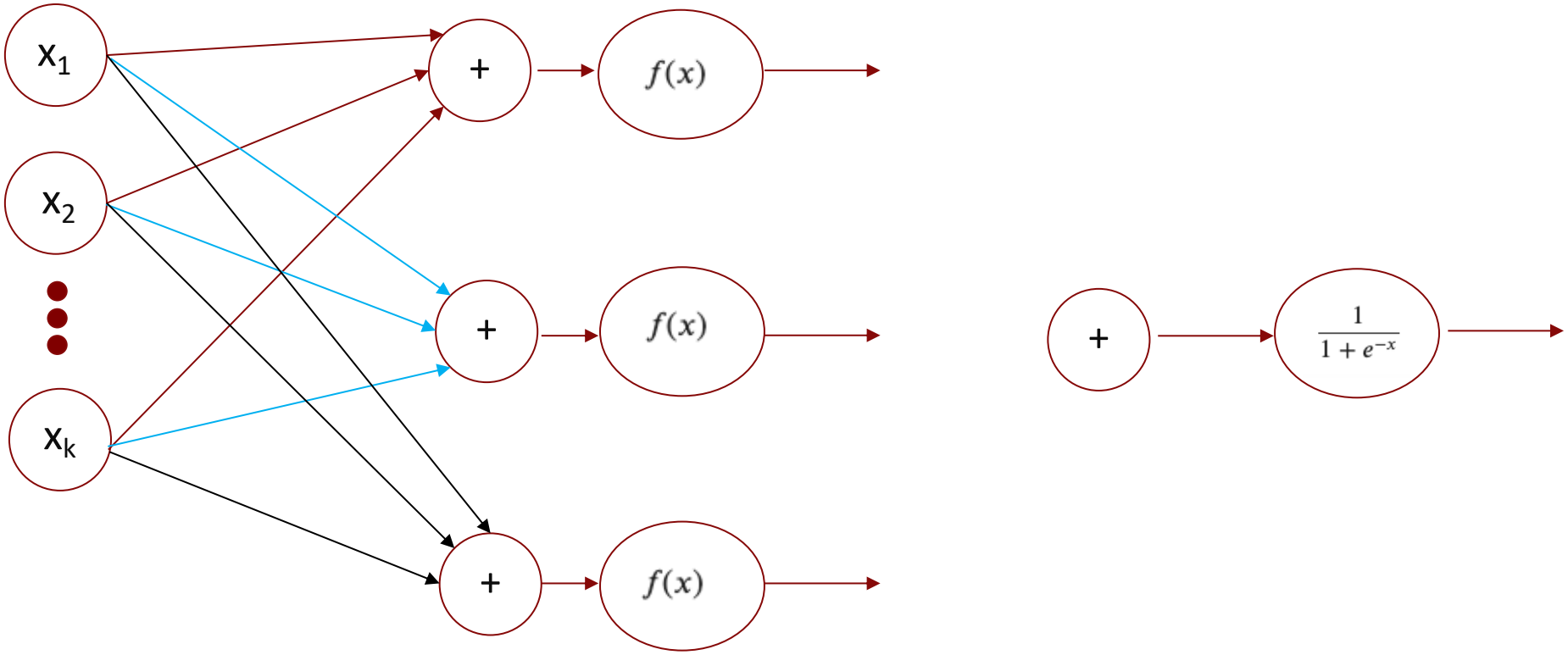


Notice that we have transformed a k-dimensional input to a 3-dimensional vector

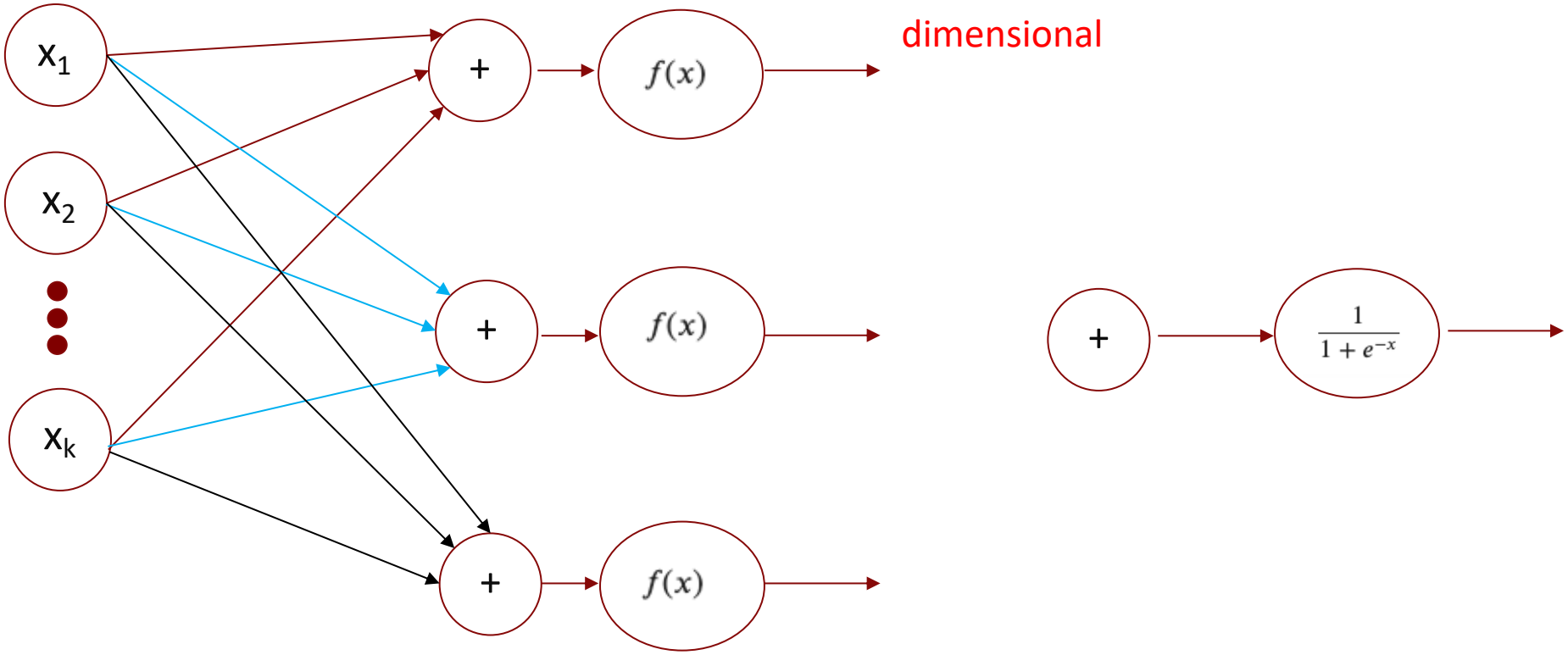


To learn smart representations, we would like to transform the inputs one or more times before we do the prediction

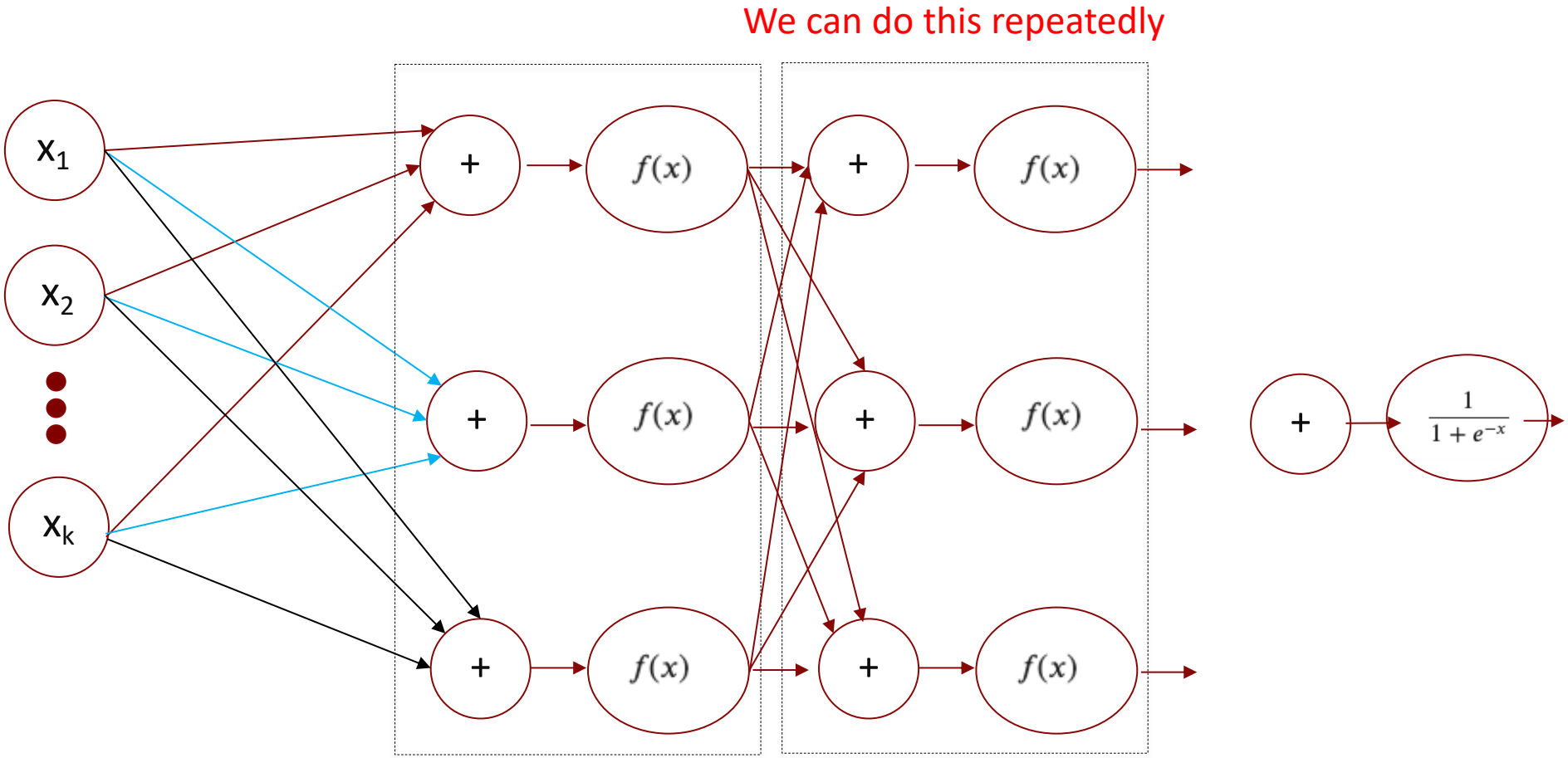
We can "flow" this 3-dimensional vector through another function



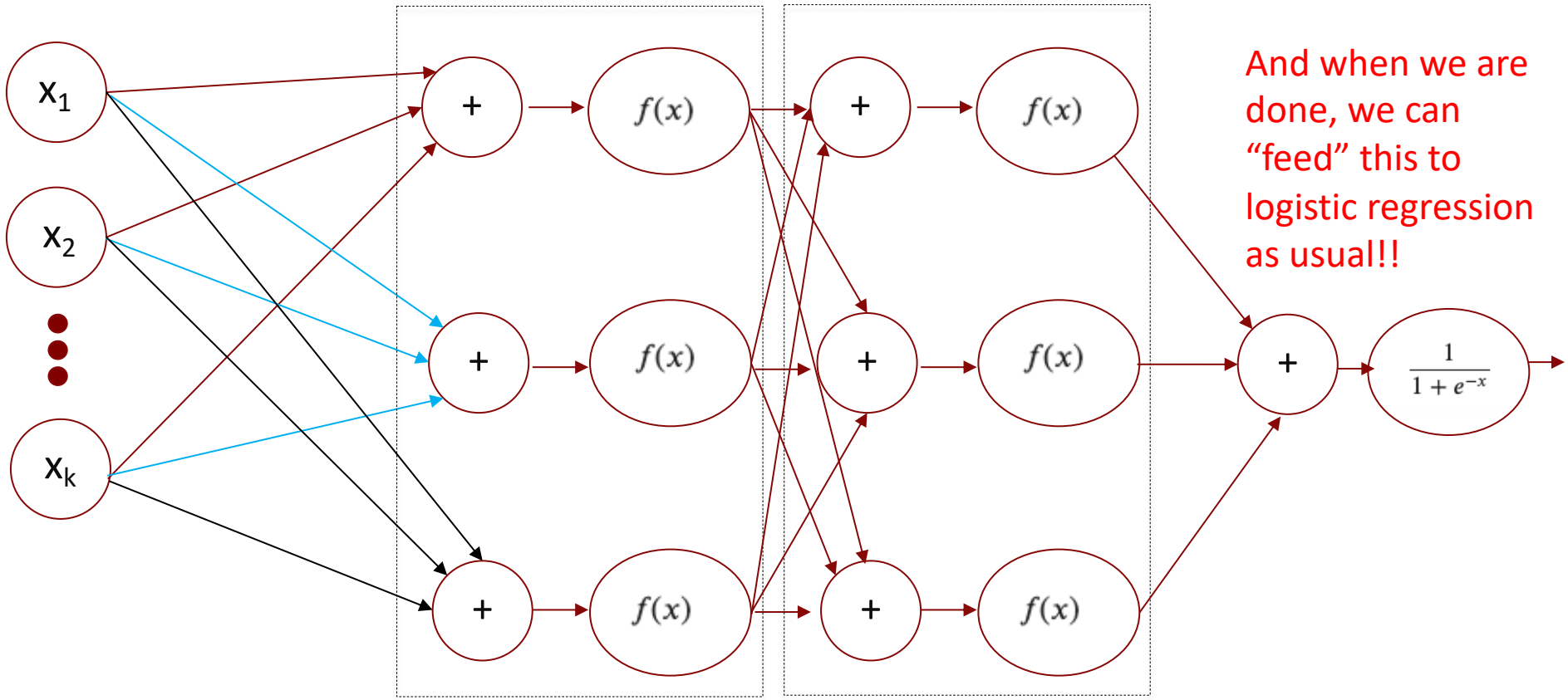
To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



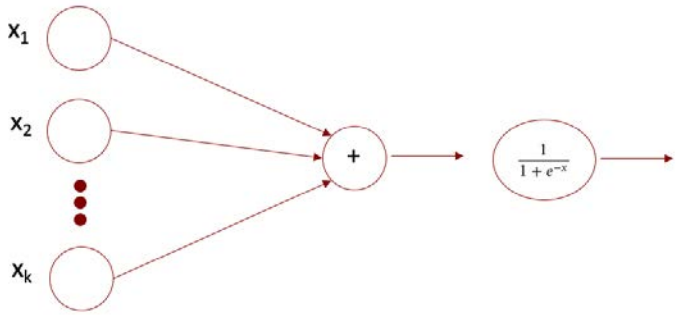
To learn smart representations, we would like to transform the inputs one or more times before we do the prediction



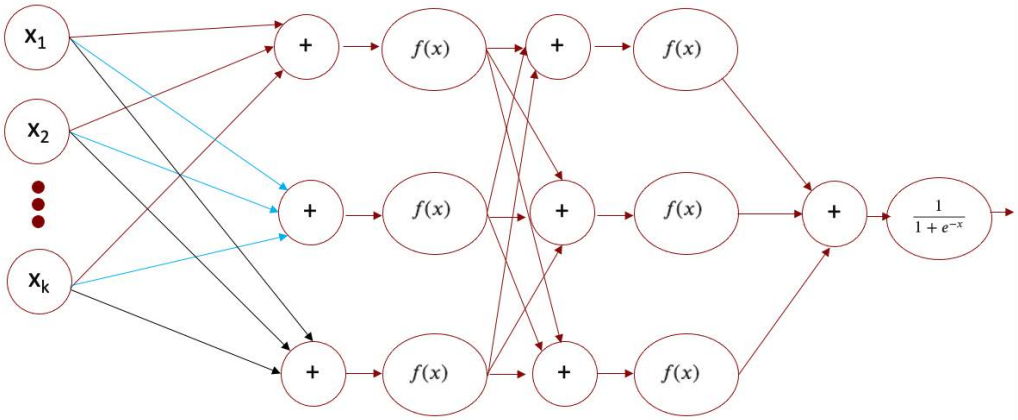
And when we are done, we can "feed" this to logistic regression as usual!!

Key Takeaway: Instead of feeding the ”raw” input to logistic regression, we feed a repeatedly transformed input

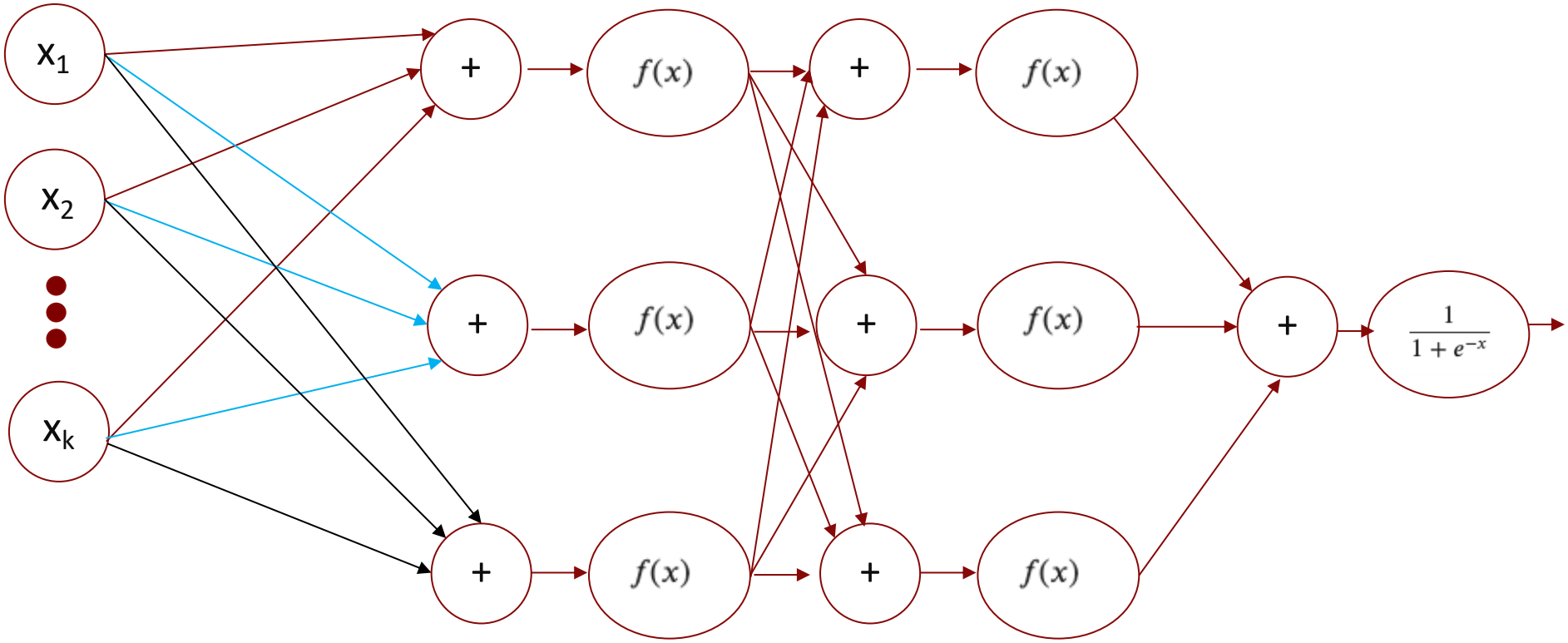
Before



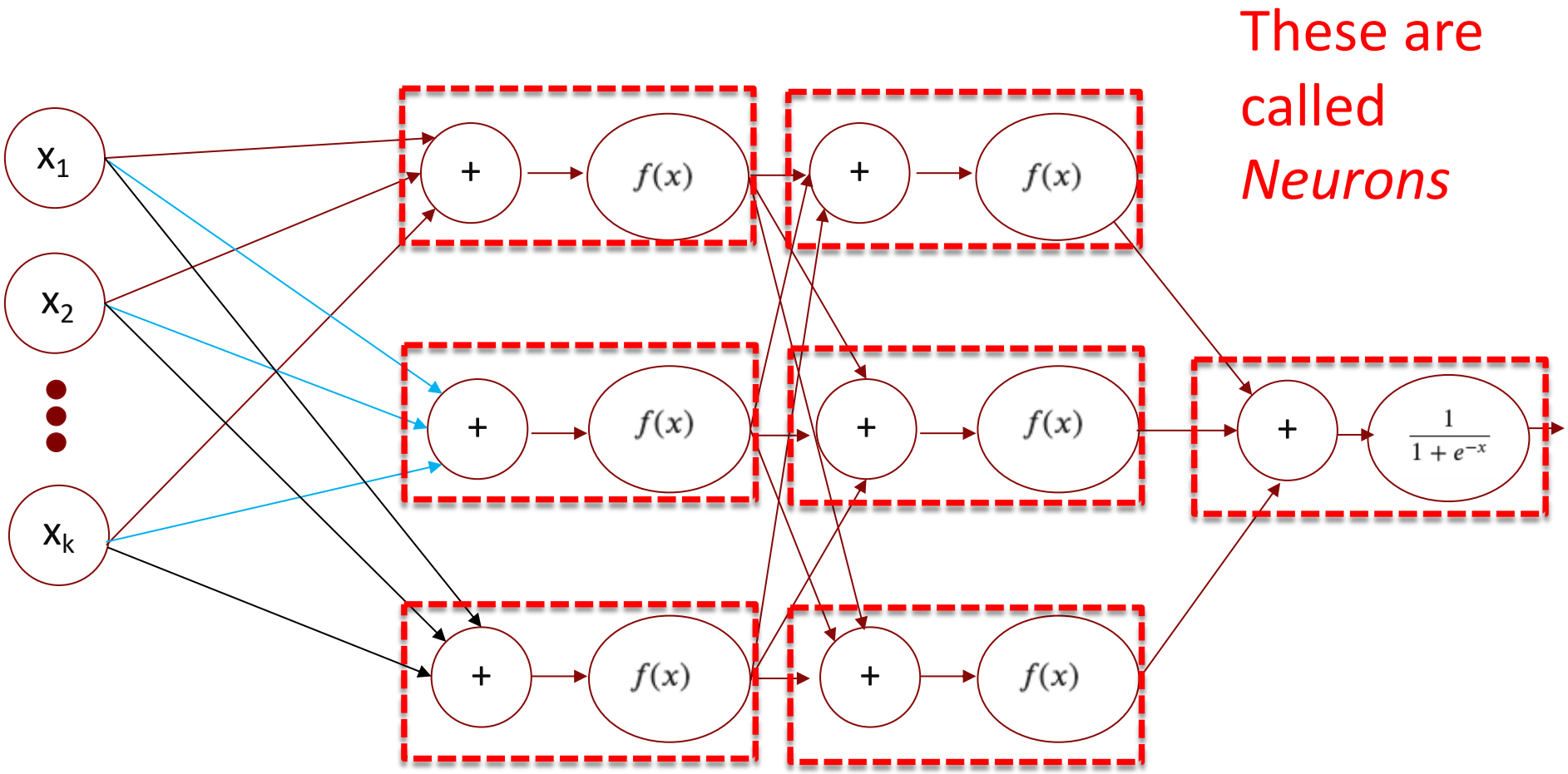
After



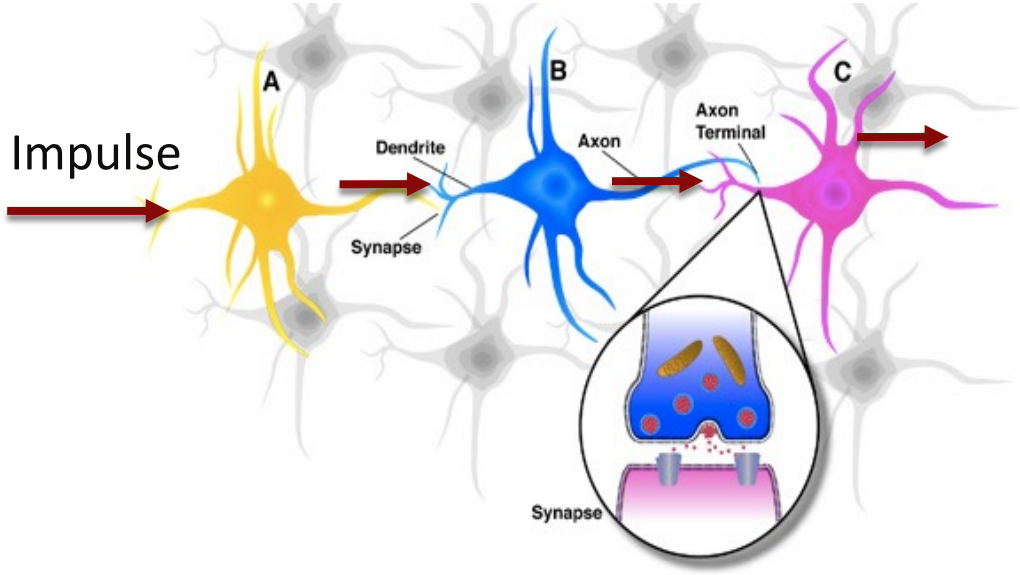
This is a Neural Network!



Terminology

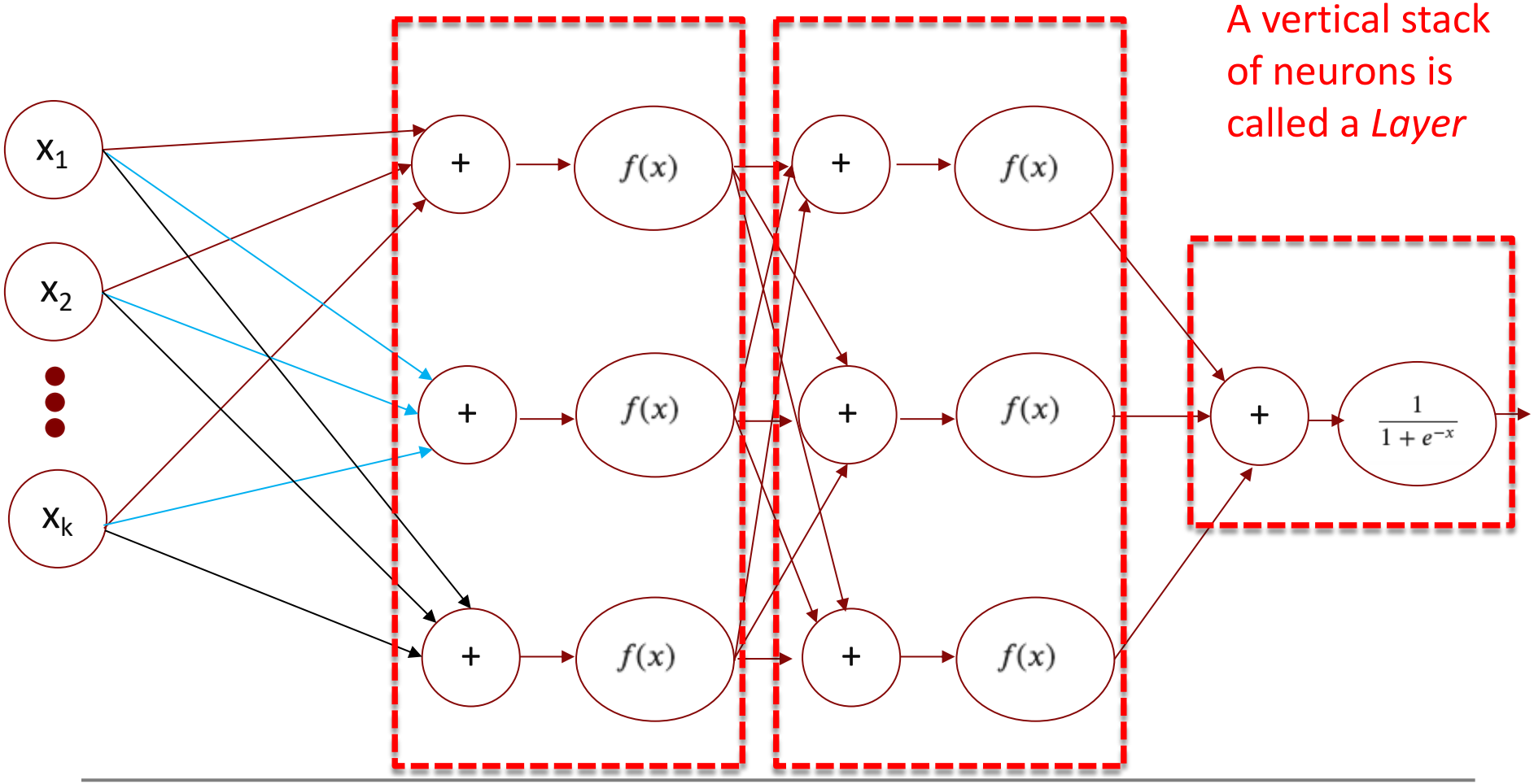


Aside: The “neural” connection

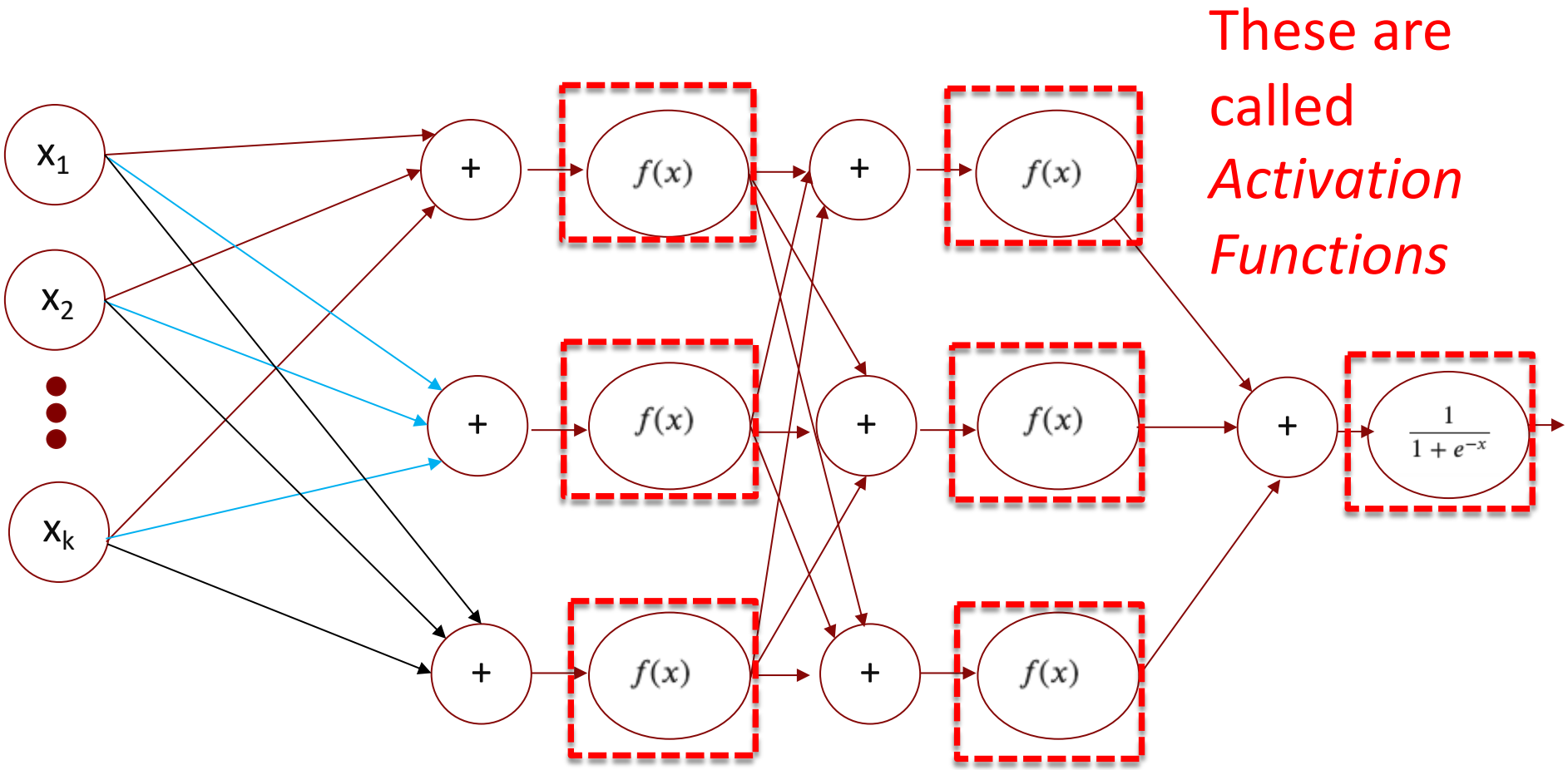


In the brain, the dendrites of one neuron connect to the axons of others at synapses, forming a network

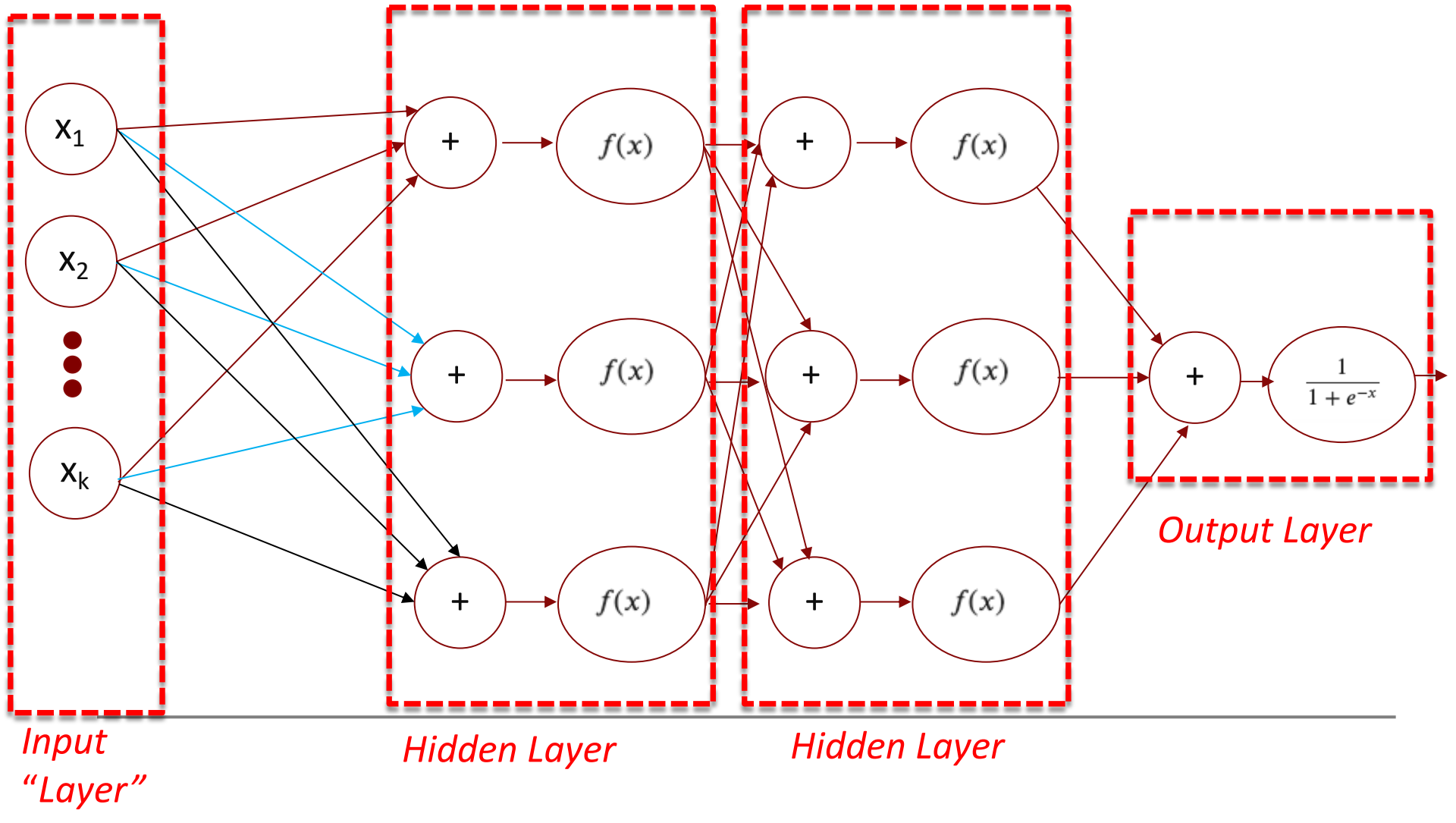
Terminology



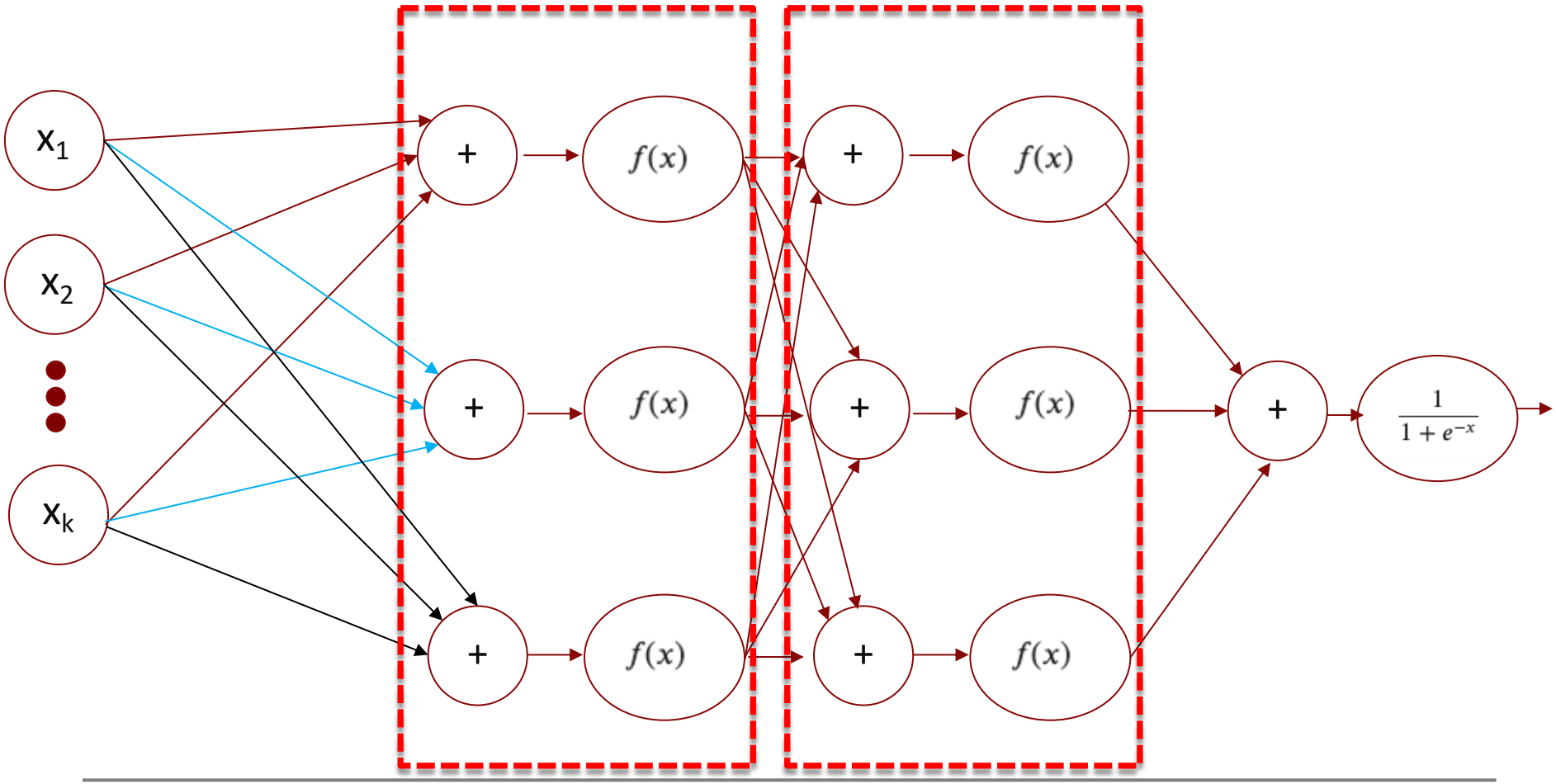
Terminology



Terminology



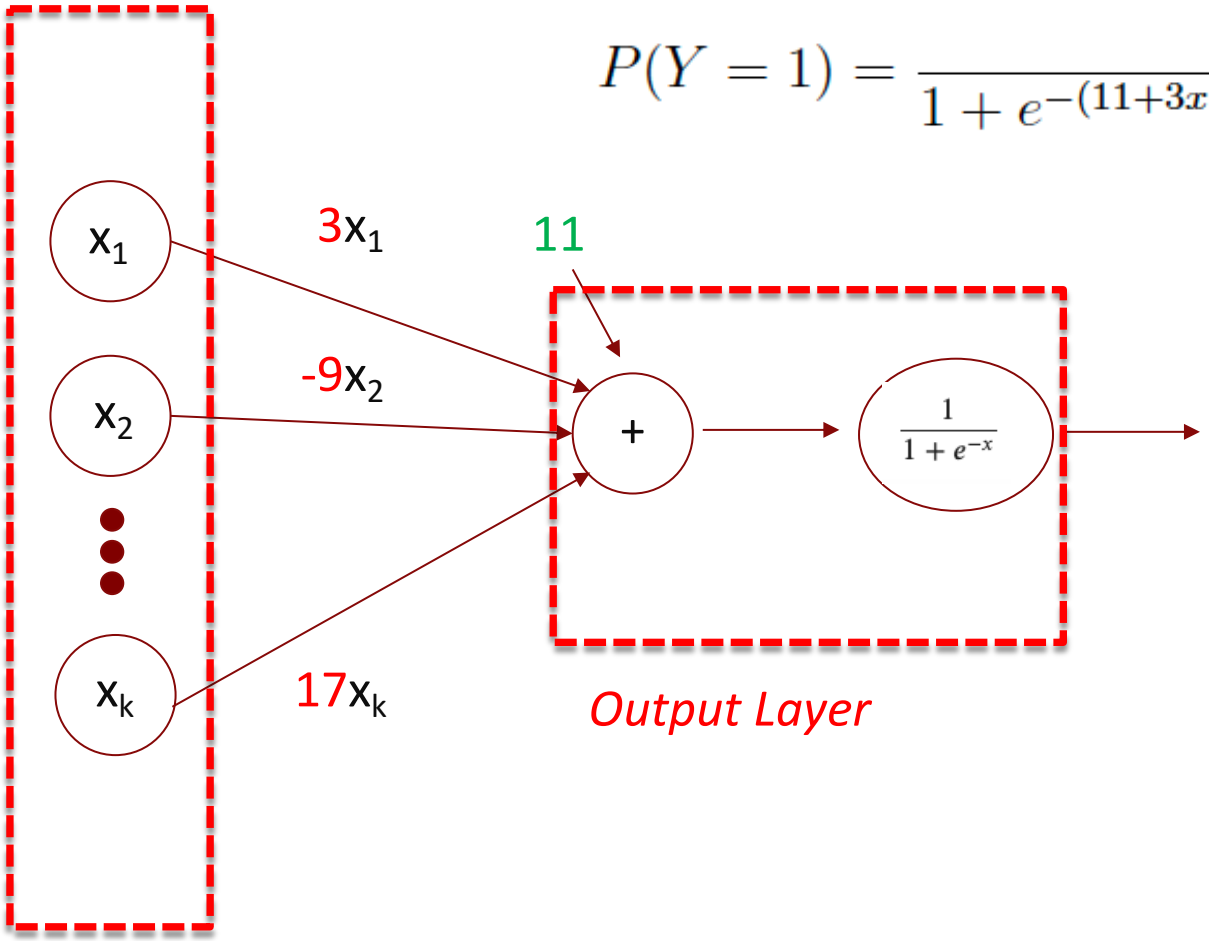
Terminology



When every neuron in a layer is connected to every neuron in the next layer, it is called Dense or Fully Connected

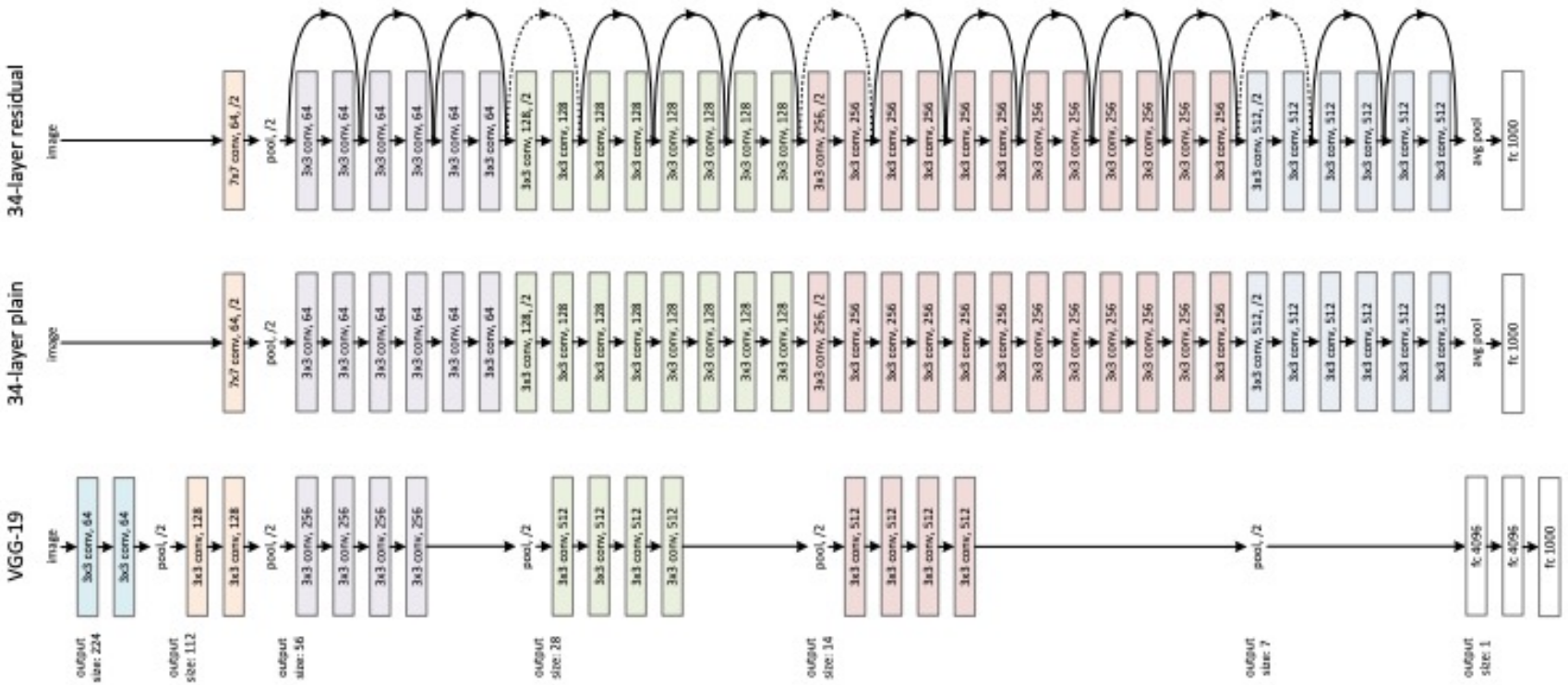
The general logistic regression model is an NN but a simple one since it has no hidden layers

$$P(Y=1) = \frac{1}{1+e^{-(11+3x_1-9x_2+\cdots+17x_k)}}$$



Deep Learning is *just* neural networks with lots and lots of …

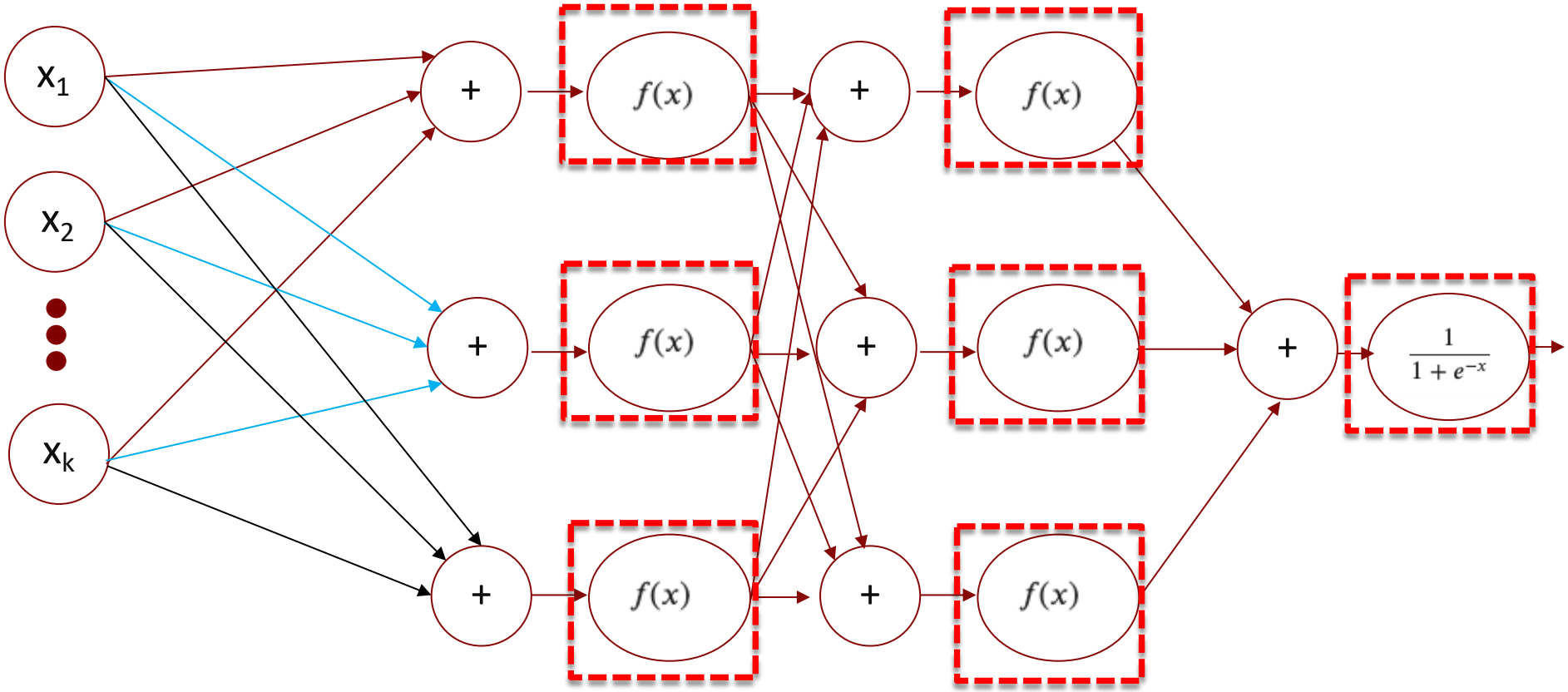
Deep Learning is *just* neural networks with lots and lots of hidden layers



© Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun/arXiv. This content is excluded from our Creative Commons license. For more information, see https://ocw.mit.edu/help/faq-fair-use/.

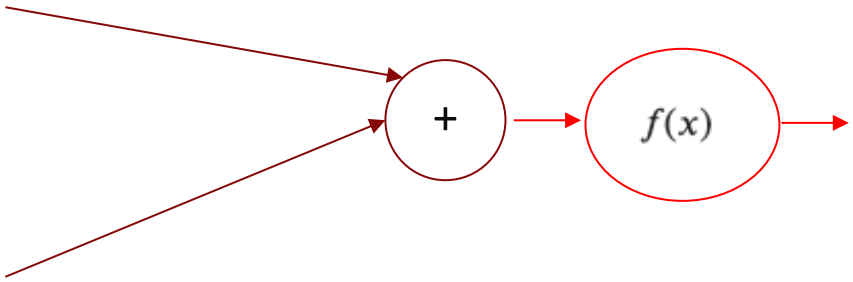
https://arxiv.org/pdf/1512.03385.pdf

Let's now turn to Activation Functions



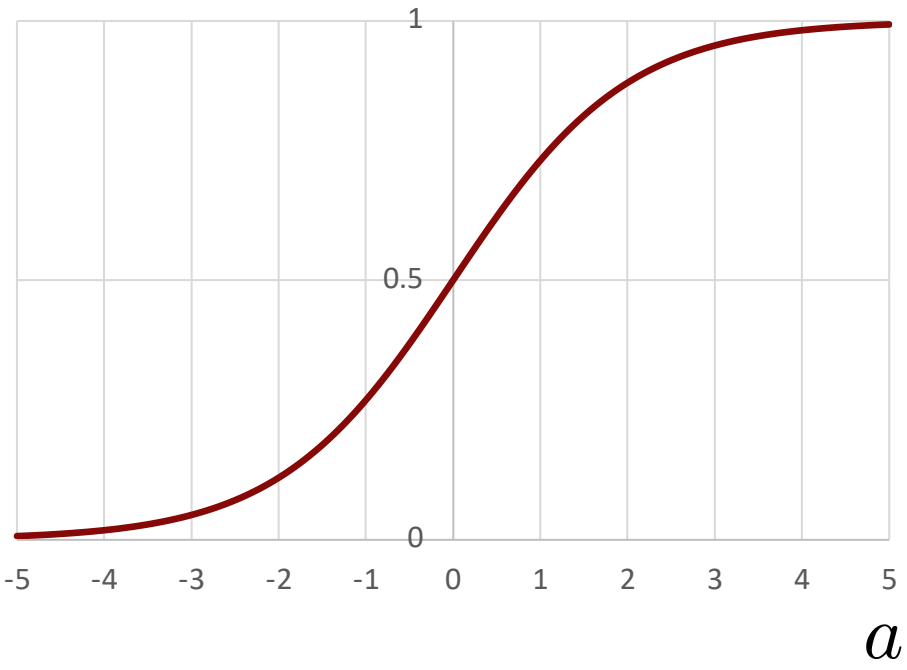
Activation functions

The activation function of a node is just a function that receives a single number and outputs a single number (i.e., scalar in -> scalar out)



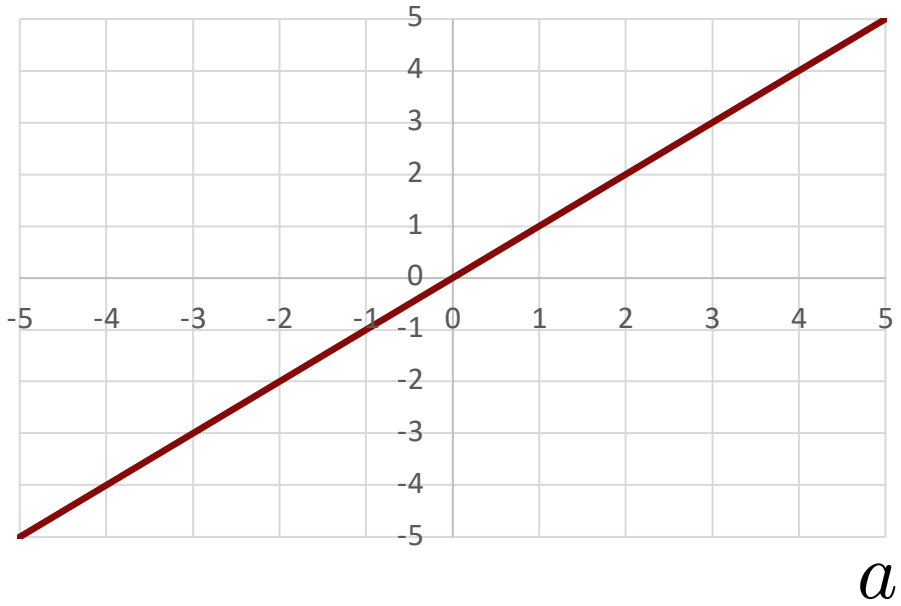
Common Activation Functions

Sigmoid activation function: $\sigma(a) = \frac{1}{1 + e^{-a}}$



Common Activation Functions

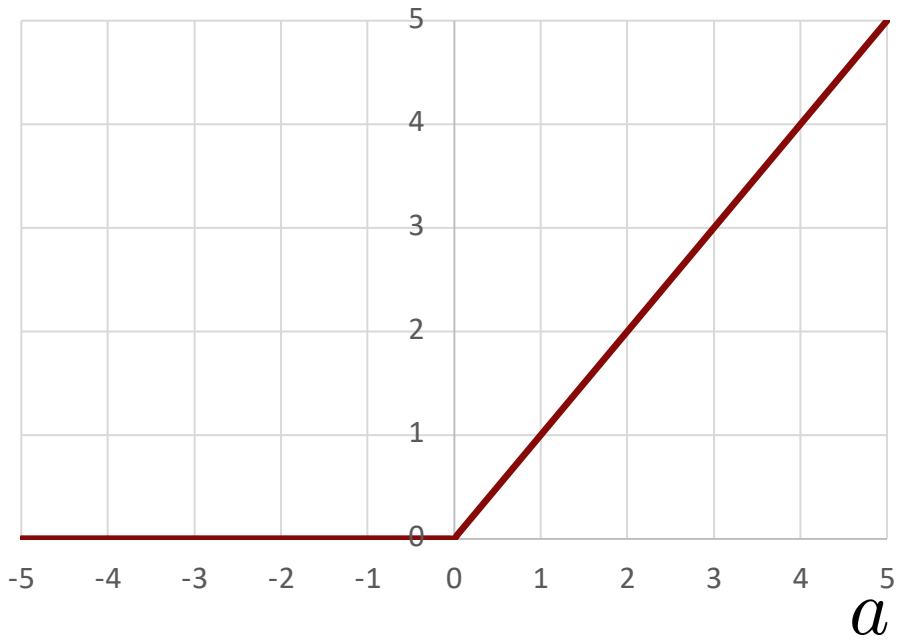
Linear activation function: $f(a) = a$



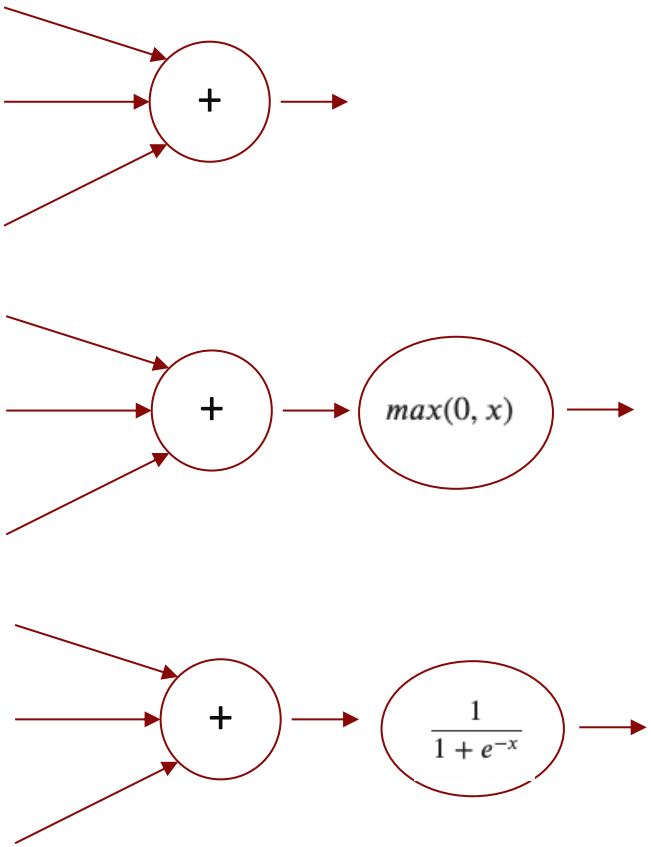
Common Activation Functions

ReLU: $g(a) = \max(0, a)$

("**Re**ctified **L**inear **U**nit")



We will use this “visual shorthand” from now on





Linear Activation

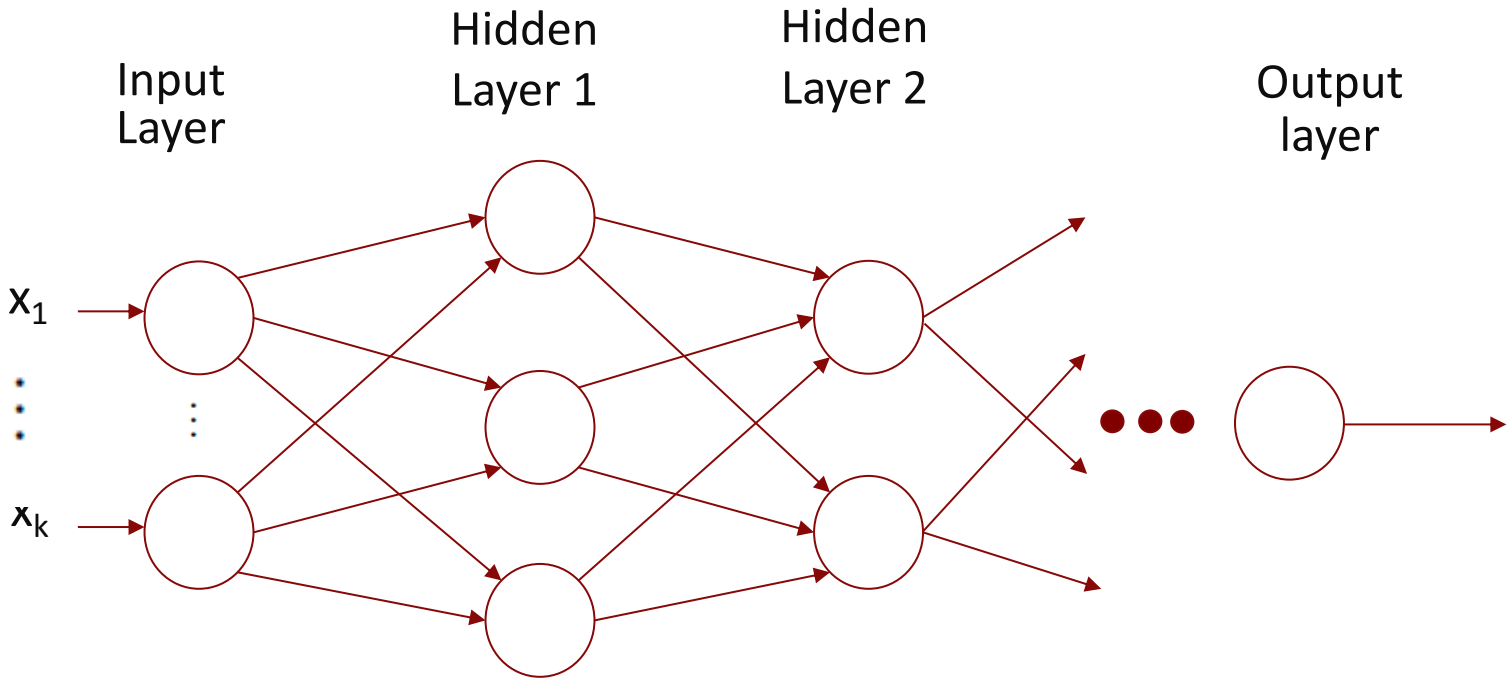


ReLU Activation



Sigmoid Activation

Recap: Designing a DNN



*User chooses the **# of hidden layers**, **# units in each layer**, the **activation function(s)** for the hidden layers and for the output layer*

Let's use a DNN for our interview classifier

- Recall the problem:
  - Two input variables (i.e., GPA and experience)
  - An output variable that should be between 0 and 1

*User chooses the **# of hidden layers**, **# units in each layer**, the **activation function(s)** for the hidden layers and for the output layer*

Let's use a DNN for our interview classifier

- Recall the problem:
  - Two input variables (i.e., GPA and experience)
  - An output variable that should be between 0 and 1
- Design choices
  - We will use **one hidden layer** with **3 neurons (ReLU)**
  - Since the output is constrained to be in (0,1), we will use the **sigmoid for the output layer**

*User chooses the **# of hidden layers**, **# units in each layer**, the **activation function(s)** for the hidden layers and for the output layer*

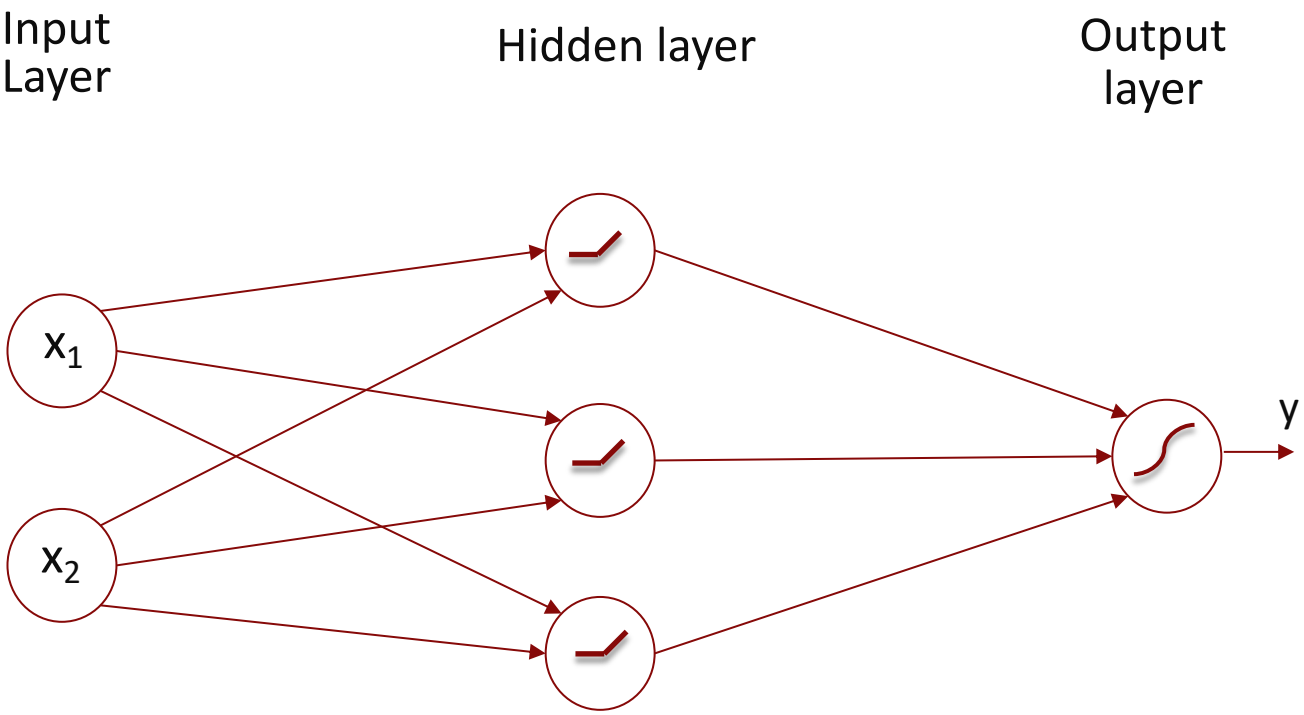
Let’s practice setting up a simple NN

Input Layer

Hidden layer

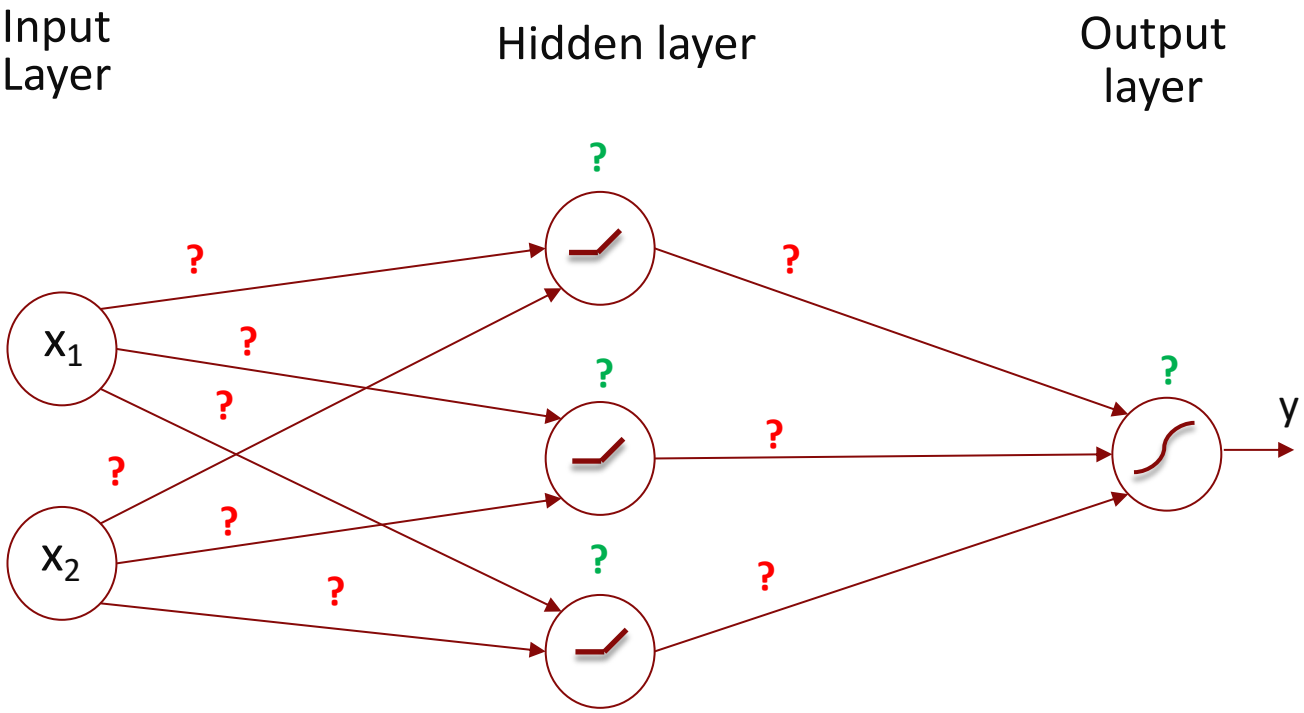
Output layer

Let's practice setting up a simple NN



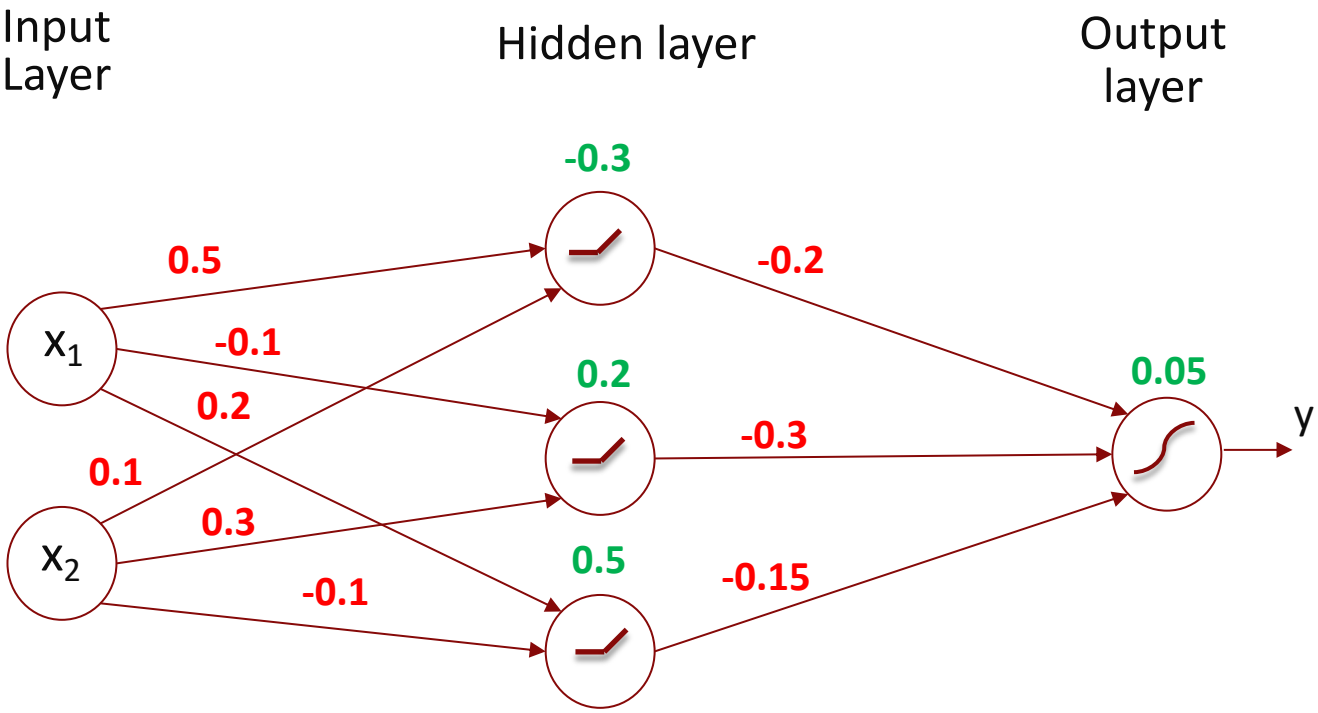
How many parameters (i.e., weights and biases) does this network have?

Let's practice setting up a simple NN



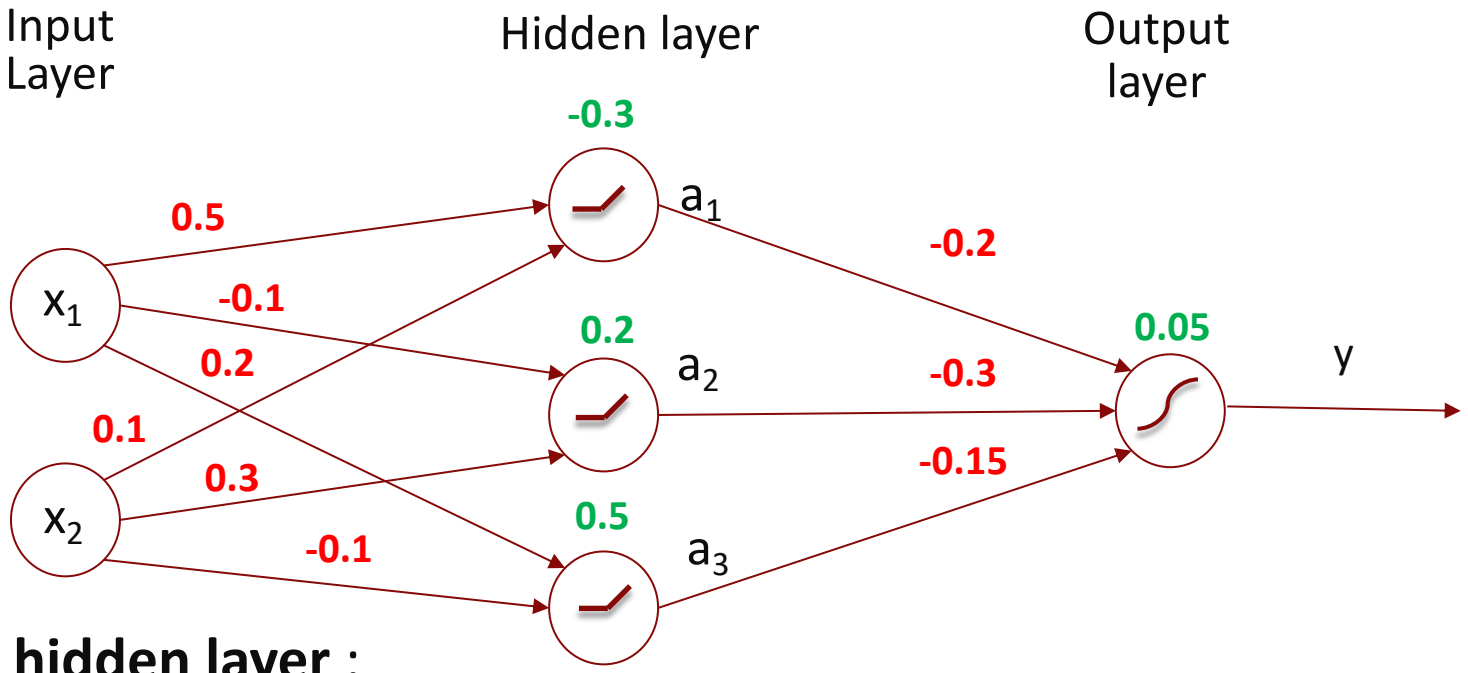
How many parameters (i.e., weights and biases) does this network have? 13

Let's assume that we have trained* this network on data and have found these values for the parameters



*details in the next class

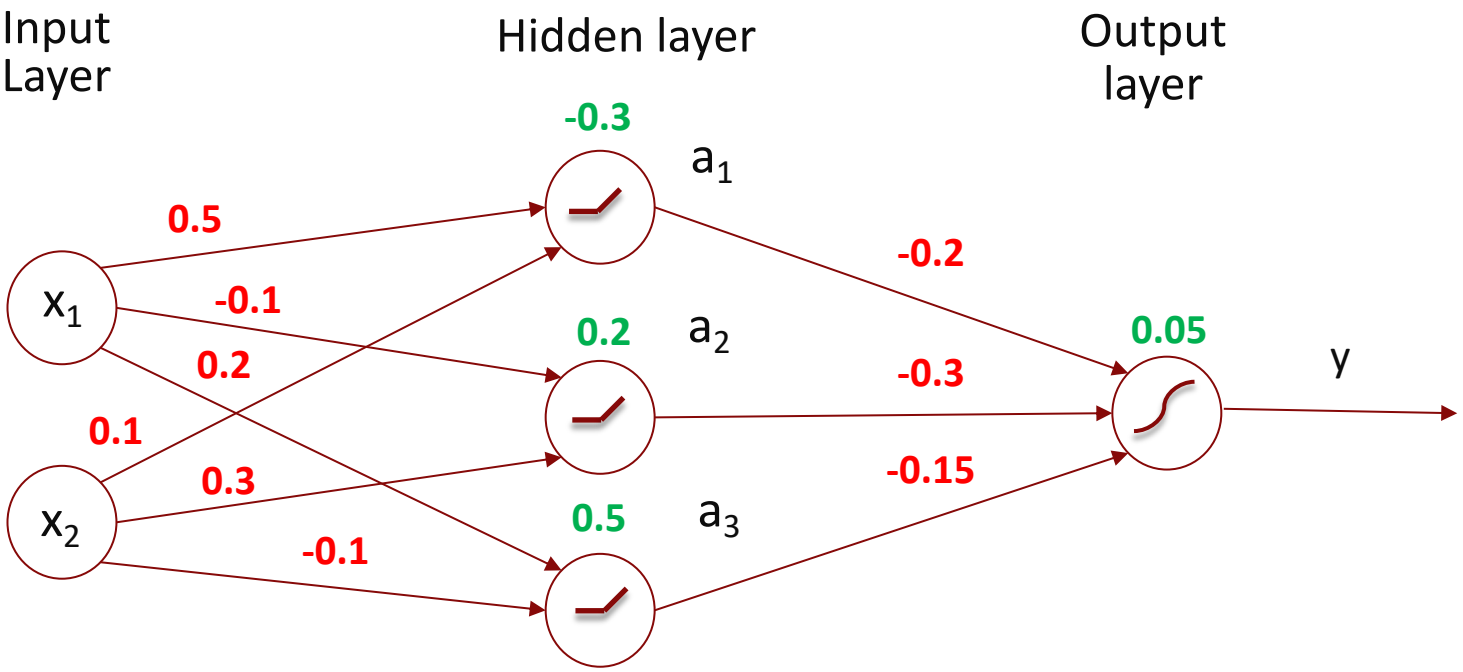
Predicting with the NN



Output of **hidden layer** :

- Top node: $\max(0, -0.3 + 0.5x_1 + 0.1x_2) = a_1$
- Middle node: $\max(0, 0.2 - 0.1x_1 + 0.3x_2) = a_2$
- Bottom node: $\max(0, 0.5 + 0.2x_1 - 0.1x_2) = a_3$

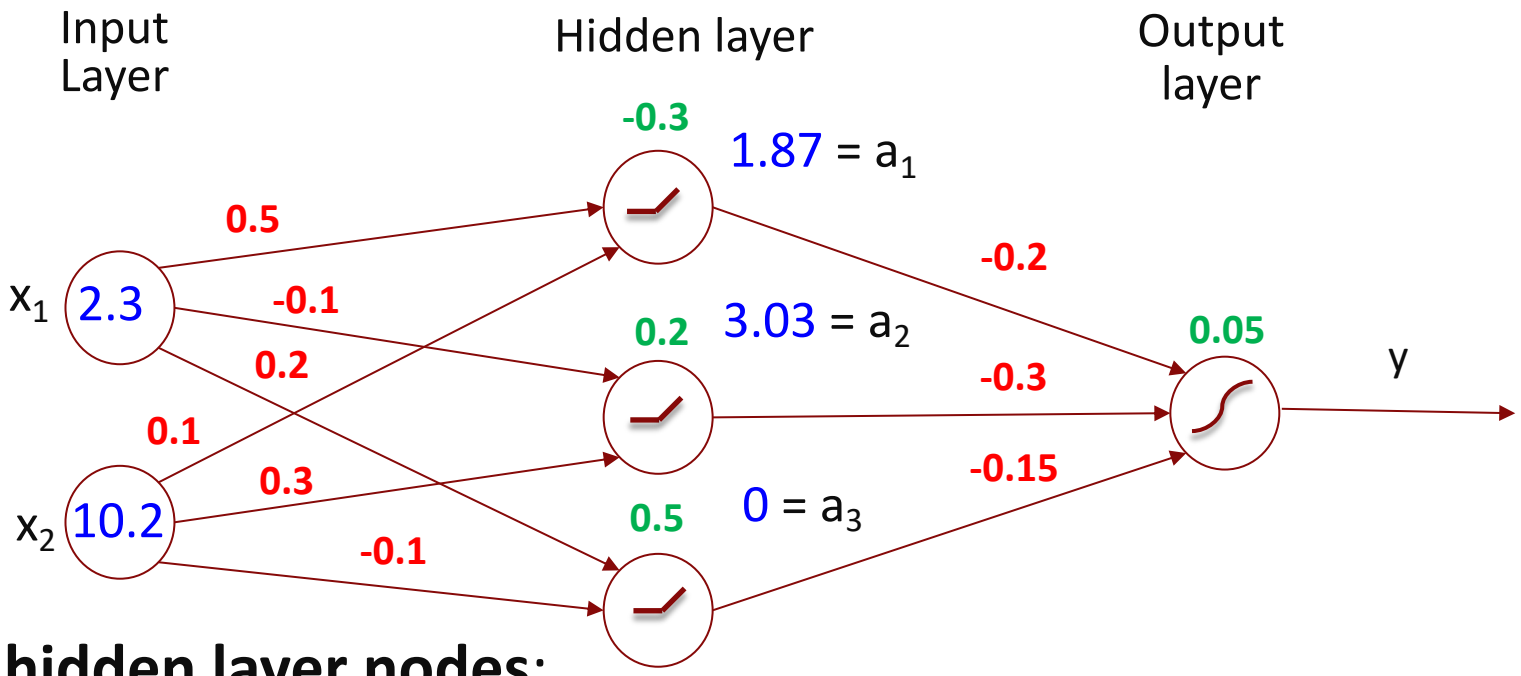
Predicting with the NN



- Recall a_1, a_2, and a_3 are the output of the hidden layer nodes
- Output of **output layer node**:

$$\frac{1}{1+e^{-(0.05-0.2a_1-0.3a_2-0.15a_3)}}$$

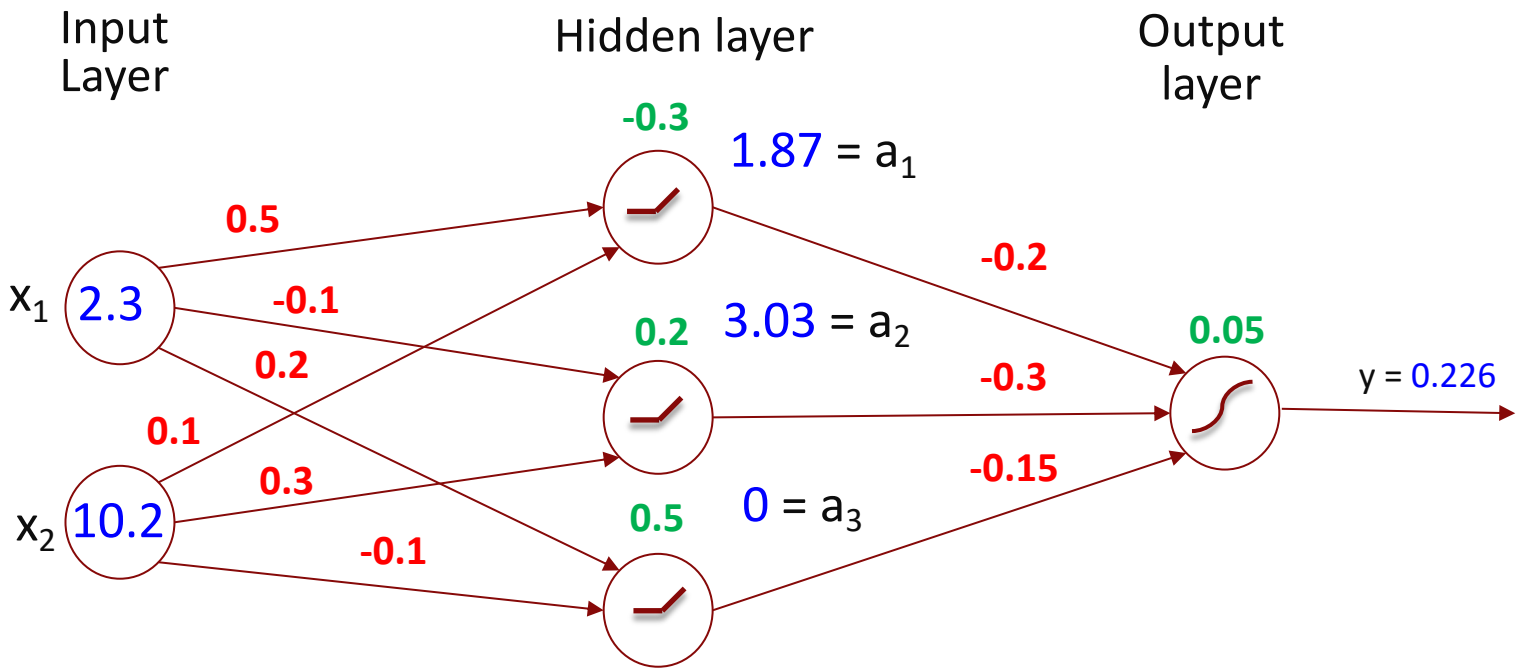
Predicting with the NN



Output of **hidden layer nodes**:

- Top node: max(0, -0.3 + 0.5*2.3 + 0.1*10.2) = 1.87 = a_1
- Middle node: max(0, 0.2 – 0.1*2.3 + 0.3*10.2) = 3.03 = a_2
- Bottom node: max(0, 0.5 + 0.2*2.3 - 0.1*10.2) = 0 = a_3

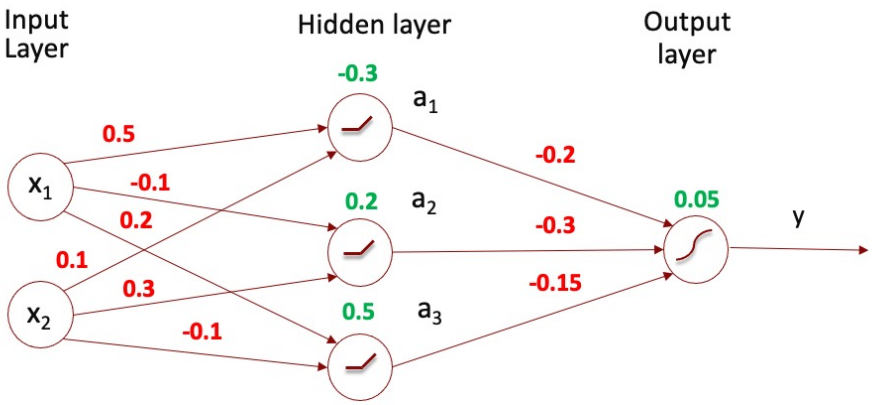
Predicting with the NN



Output of **output layer node**:

$$\frac{1}{1+e^{-(0.05-0.2*1.87-0.3*3.03-0.15*0)}} = 0.226$$

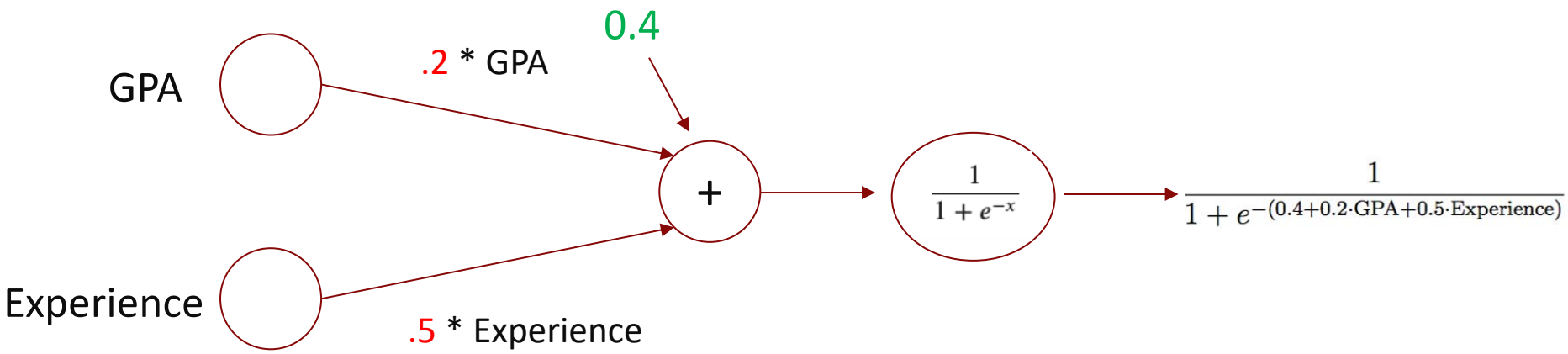
The Network can be written as this function



Equivalent

$$y = \frac{1}{1 + e^{-(.05 - 0.2(max(0, -0.3 + 0.5x_1 + 0.1x_2)) - 0.3(max(0, 0.2 - 0.1x_1 + 0.3x_2) - 0.15(max(0.5 + 0.2x_1 - 0.1x_2))}}$$

Contrast with the Logistic Regression model from before



Equivalent

$$P(Y=1) = \frac{1}{1+e^{-(0.4+0.2\cdot\text{GPA}+0.5\cdot\text{Experience})}}$$

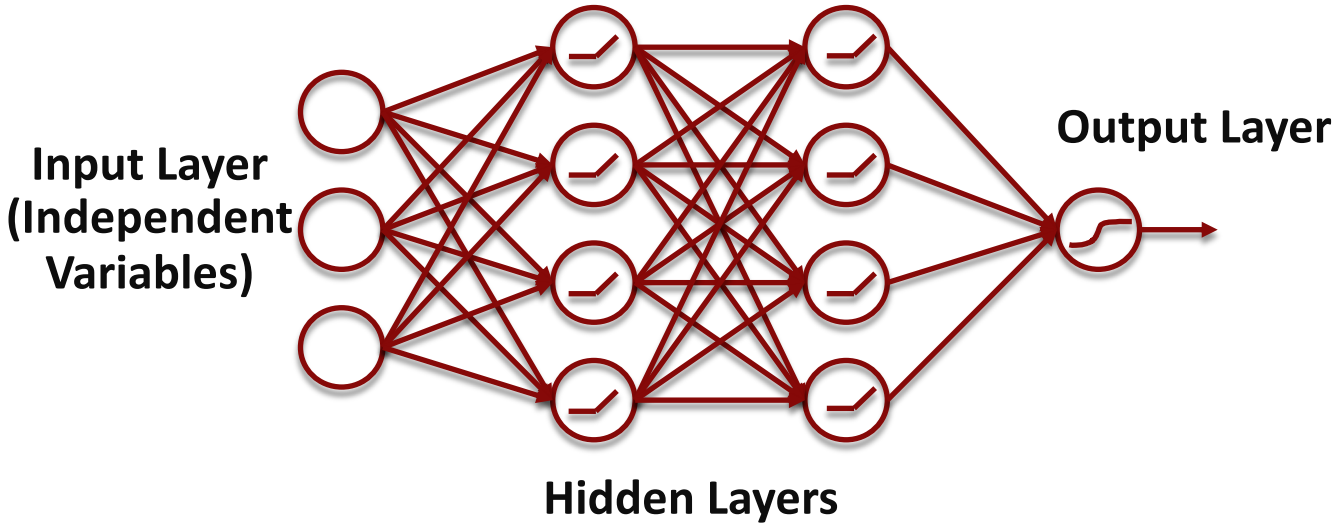
Note the complexity of even this simple network compared to the logistic regression model

$$y = \frac{1}{1 + e^{-(.05-0.2(max(0,-0.3+0.5x_1+0.1x_2))-0.3(max(0,0.2-0.1x_1+0.3x_2)-0.15(max(0.5+0.2x_1-0.1x_2))}}$$

$$y = \frac{1}{1 + e^{-(0.4+0.2x_1+0.5x_2)}}$$

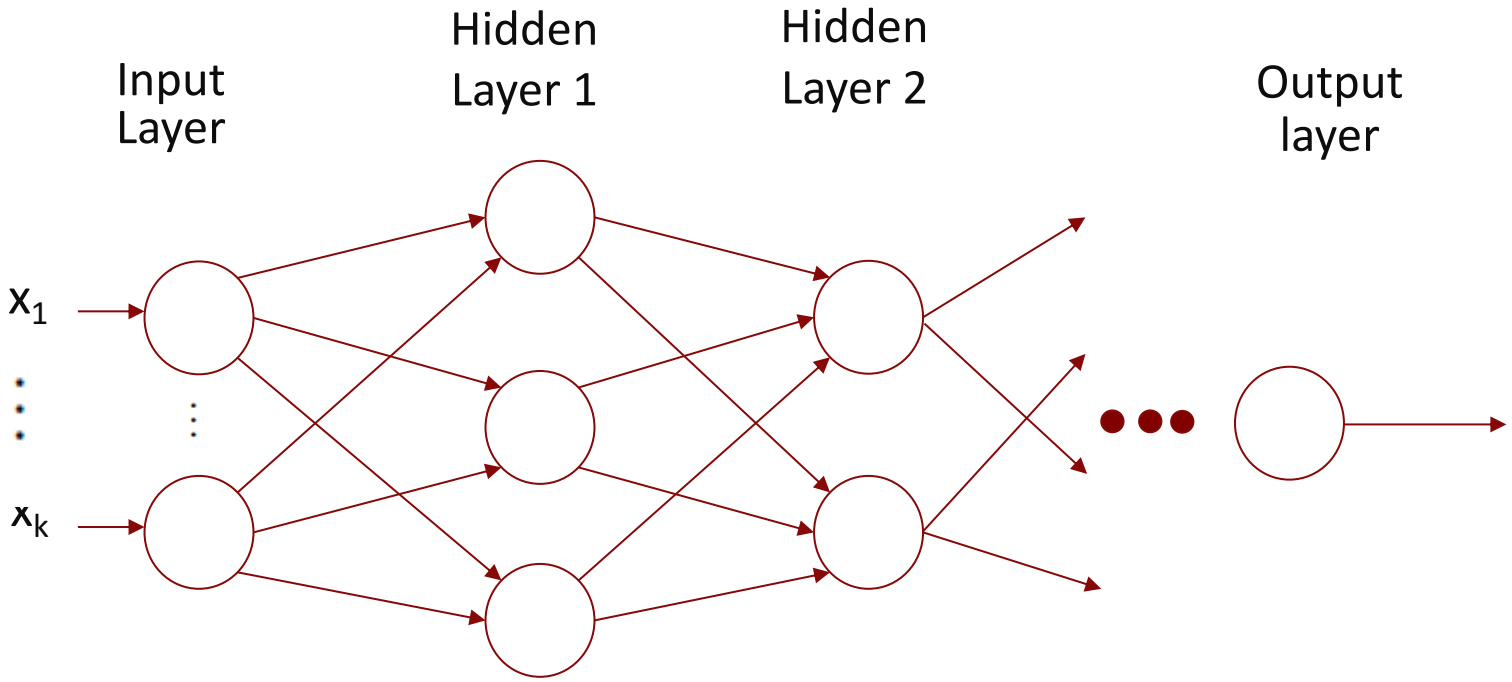
y is a much more complex function of its inputs x_1 and x_2 compared to (say) $\frac{1}{1 + e^{-(0.4+0.2 \cdot \text{GPA}+0.5 \cdot \text{Experience})}}$ and it can capture more complex relationships between *x* and *y*.

Summary: A Deep Neural Network



- This is a **feedforward** (or **vanilla**) neural network
- In general, the arrangement of neurons into layers, the activation functions, and the connections between layers are referred to as the network's **architecture**

Summary: Designing a DNN



*User chooses the **# of hidden layers**, **# units in each layer**, the **activation function(s)** for the hidden layers and for the output layer*

Source and license

This lecture adapts material from:

Vivek Farias and Rama Ramakrishnan, Hands-On Deep Learning,
MIT OpenCourseWare 15.773, Spring 2024.

Source:

ocw.mit.edu/courses/15-773-hands-on-deep-learning-spring-2024/

License: Creative Commons Attribution-NonCommercial-ShareAlike 4.0
creativecommons.org/licenses/by-nc-sa/4.0/

Adapted for ECE116 at UC San Diego, Fall 2026.
Changes include course framing, sequencing, omissions, and ECE116 branding.